Lab 3: Reinforcement Learning

Your Name

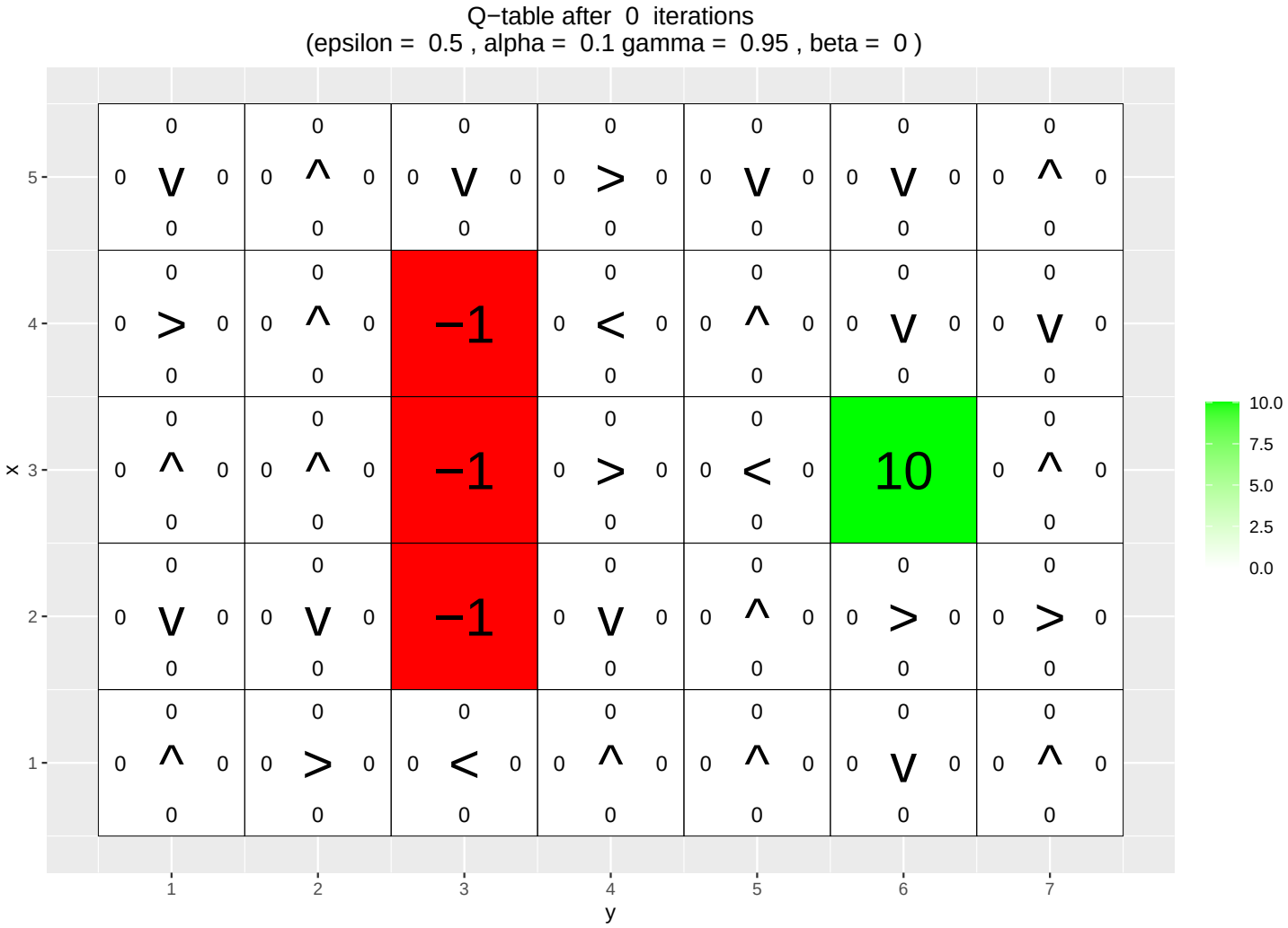
2026-10-04

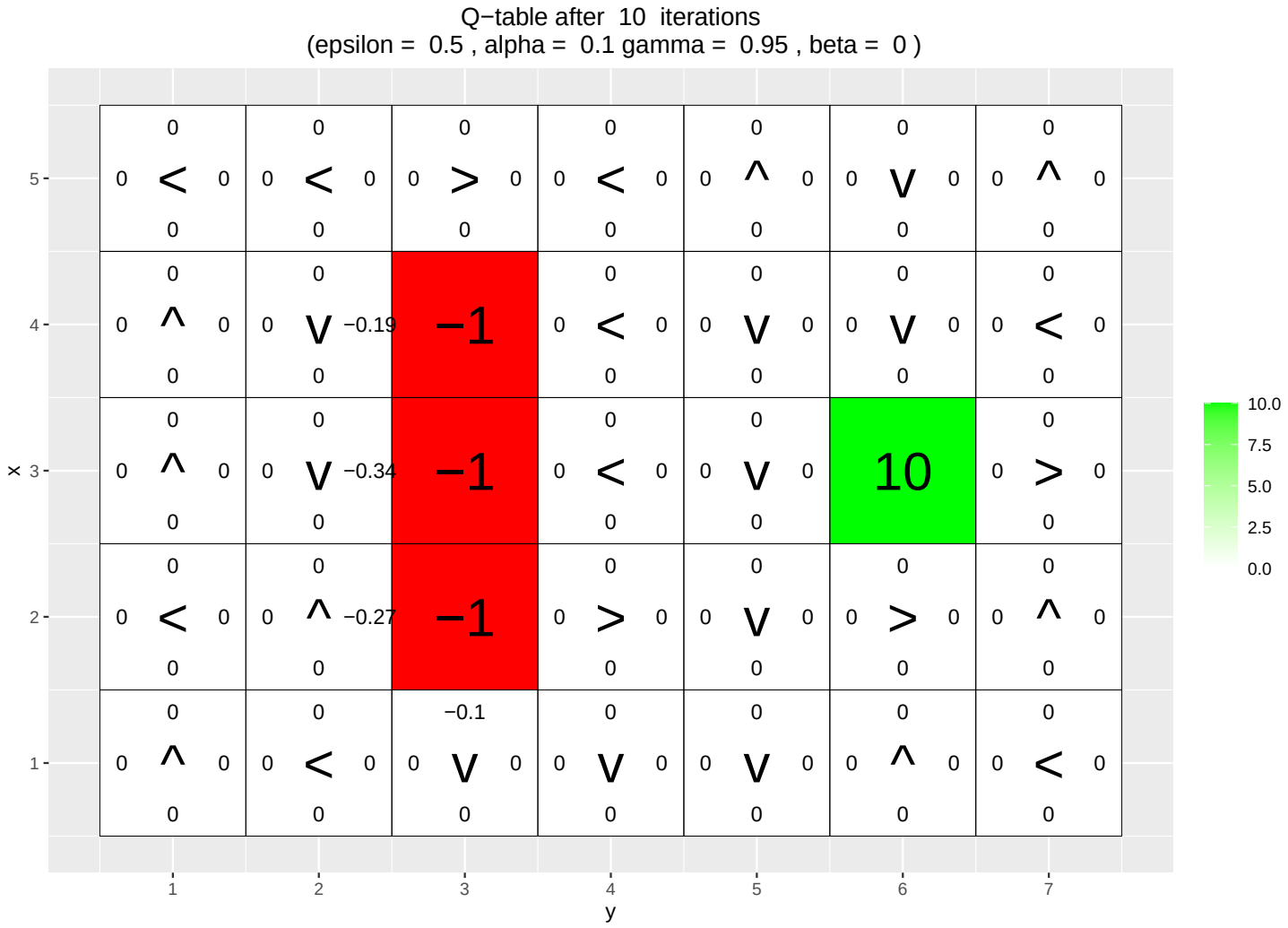
1. Q-learning

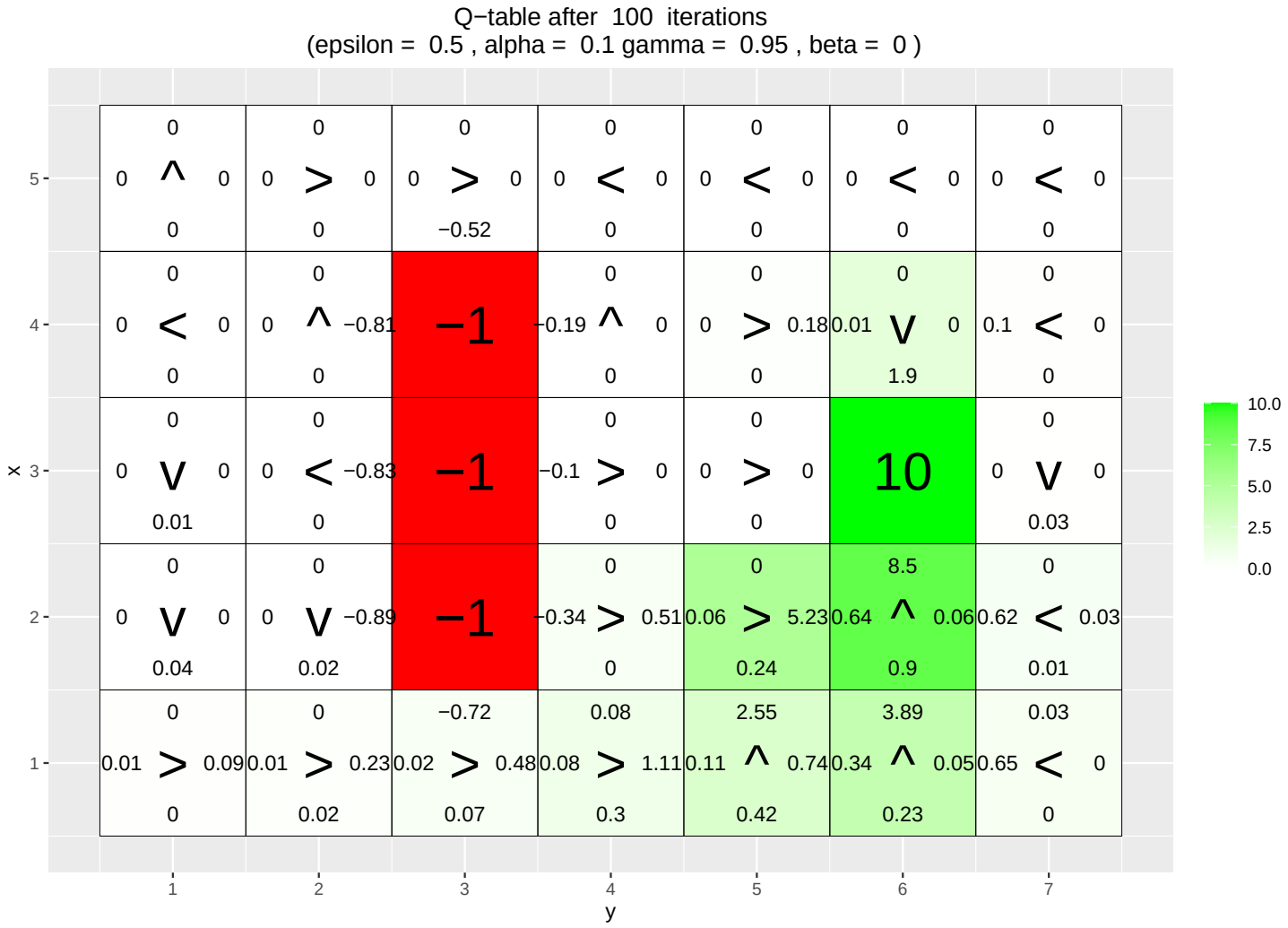
The grid-world is a fully observable, episodic Markov decision process. The agent receives the reward of the state it enters, and any non-zero-reward state ends the episode. The Q-learning implementation, including the epsilon-greedy policy, random tie-breaking, transition model and temporal-difference update, is listed in Appendix A.1.

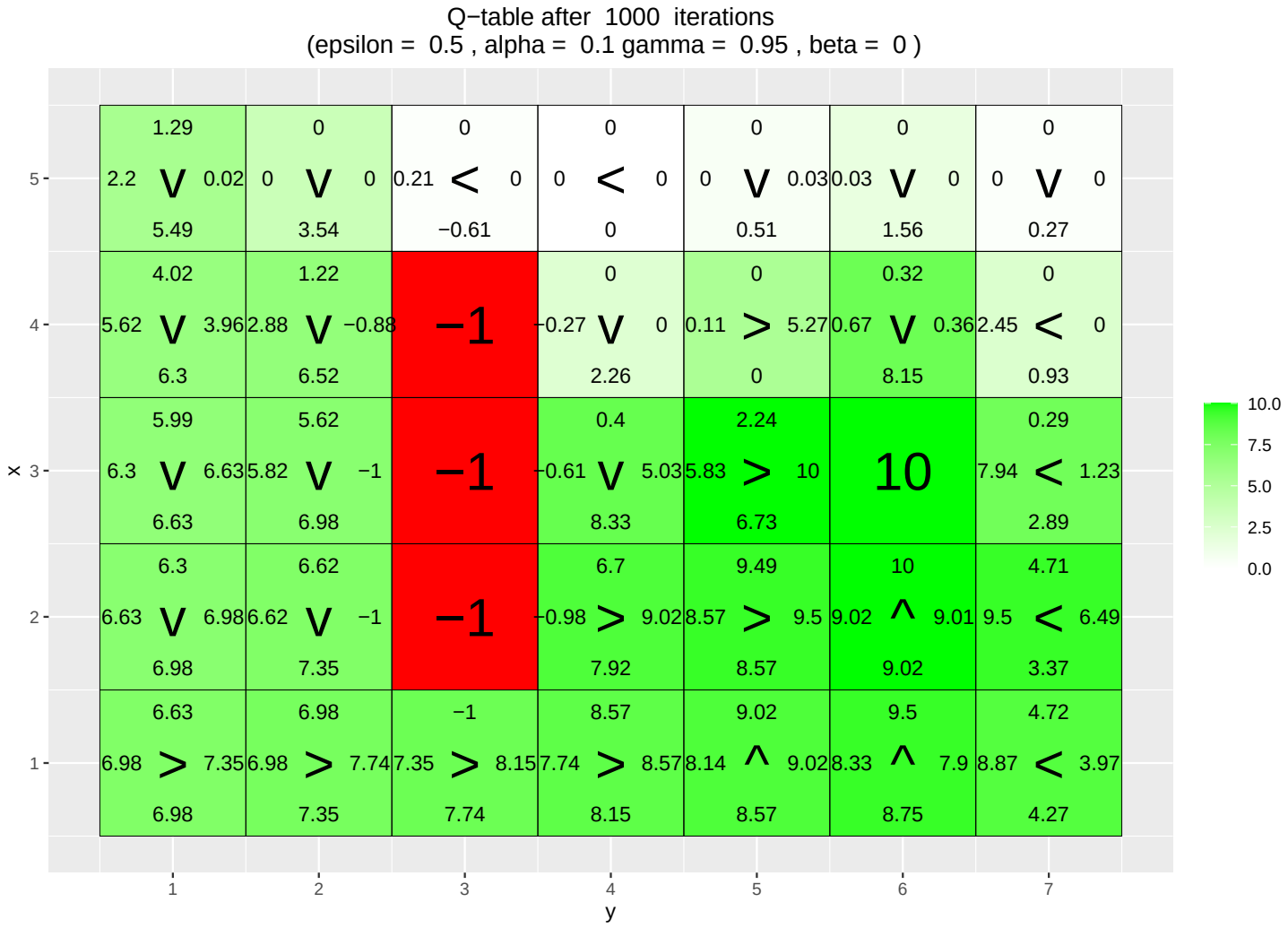
1.1 Environment A

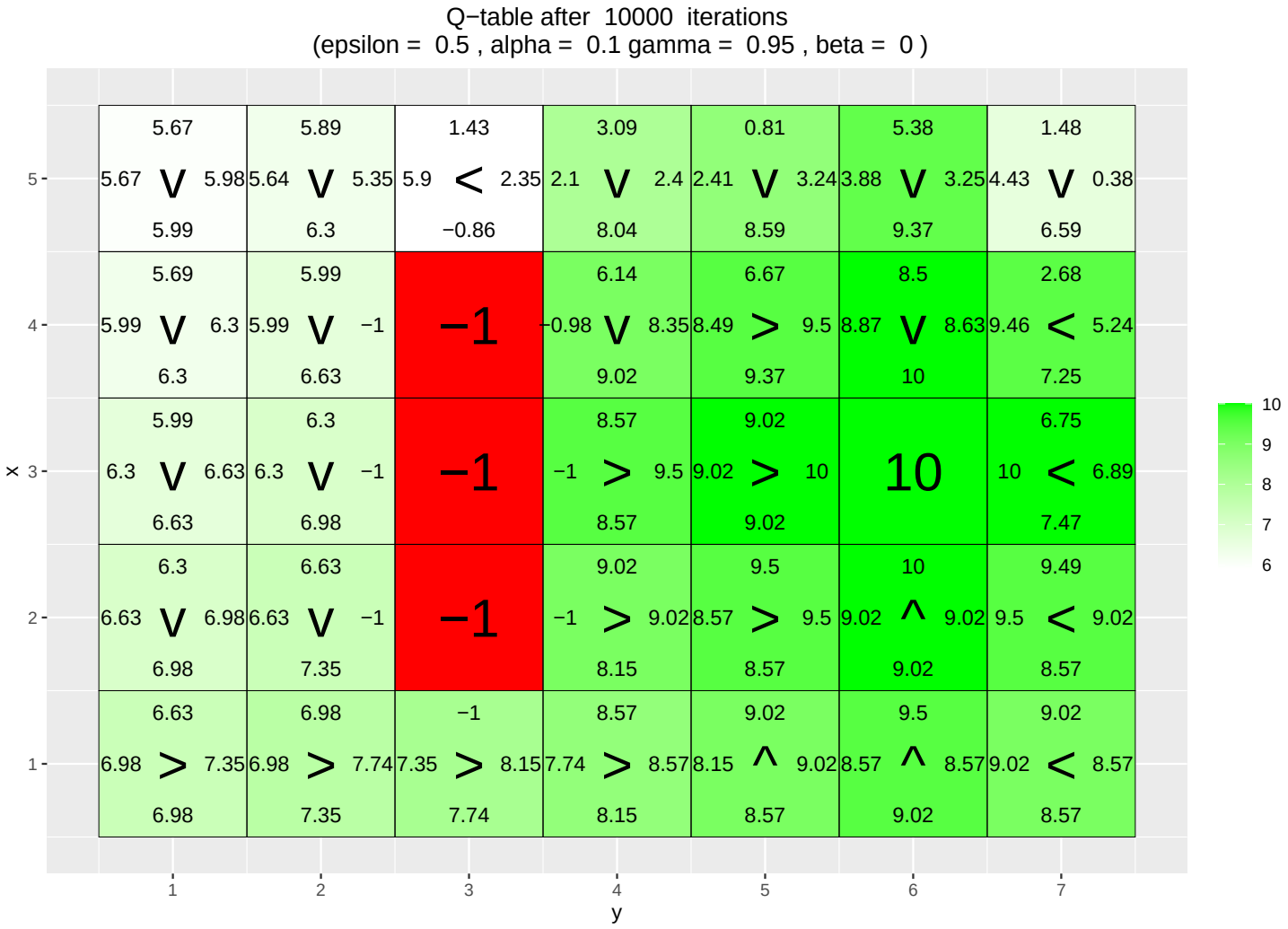
The agent starts each episode at state (3, 1). The goal reward is +10 at (3, 6), and states (2, 3), (3, 3) and (4, 3) have reward -1. The Q-table and learned greedy policy after training are shown below.




Q−table after 10 iterations
(epsilon = 0.5 , alpha = 0.1 gamma = 0.95 , beta = 0)
x
y
1
2
3
4
5
6
7
−0.19
−0.34
−0.27
−0.1
−1
10
10.0
7.5
5.0
2.5
0.0


Q−table after 100 iterations
(epsilon = 0.5 , alpha = 0.1 gamma = 0.95 , beta = 0)
x
y
1
2
3
4
5
6
7
10.0
7.5
5.0
2.5
0.0
−1
10


Q−table after 1000 iterations
(epsilon = 0.5 , alpha = 0.1 gamma = 0.95 , beta = 0)
1.29
2.2 V 0.02
5.49
0
0 V 0
3.54
0
0.21 < 0
−0.61
0
0 < 0
0
0
0 V 0.03
0.51
0
0.03 V 0
1.56
0
0 V 0
0.27
4.02
5.62 V 3.96
6.3
1.22
2.88 V −0.88
6.52
−1
0
−0.27 V 0
2.26
0
0.11 > 5.27
0
0.32
0.67 V 0.36
8.15
0
2.45 < 0
0.93
5.99
6.3 V 6.63
6.63
5.62
5.82 V −1
6.98
−1
0.4
−0.61 V 5.03
8.33
2.24
5.83 > 10
6.73
10
0.29
7.94 < 1.23
2.89
6.3
6.63 V 6.98
6.98
6.62
6.62 V −1
7.35
−1
6.7
−0.98 > 9.02
7.92
9.49
8.57 > 9.5
8.57
10
9.02 ^ 9.01
9.02
4.71
9.5 < 6.49
3.37
6.63
6.98 > 7.35
6.98
6.98
6.98 > 7.74
7.35
−1
7.35 > 8.15
7.74
8.57
7.74 > 8.57
8.15
9.02
8.14 ^ 9.02
8.57
9.5
8.33 ^ 7.9
8.75
4.72
8.87 < 3.97
4.27
x
y
1
2
3
4
5
6
7
10.0
7.5
5.0
2.5
0.0



What has the agent learned after the first 10 episodes?

After only 10 episodes, learning is preliminary: Q-values have been updated mainly for state-action pairs encountered along the sampled trajectories. The greedy policy therefore gives a partial picture of how to reach the +10 reward; values for less-visited actions and states may still be near their initial value of zero.

Is the final greedy policy optimal for all states? Why or why not?

Not necessarily. Training always starts at (3, 1), and 10,000 episodes do not guarantee that every state-action pair is explored enough to learn its optimal value. The policy is good on commonly visited routes but not optimal from every state, top left corner for example.

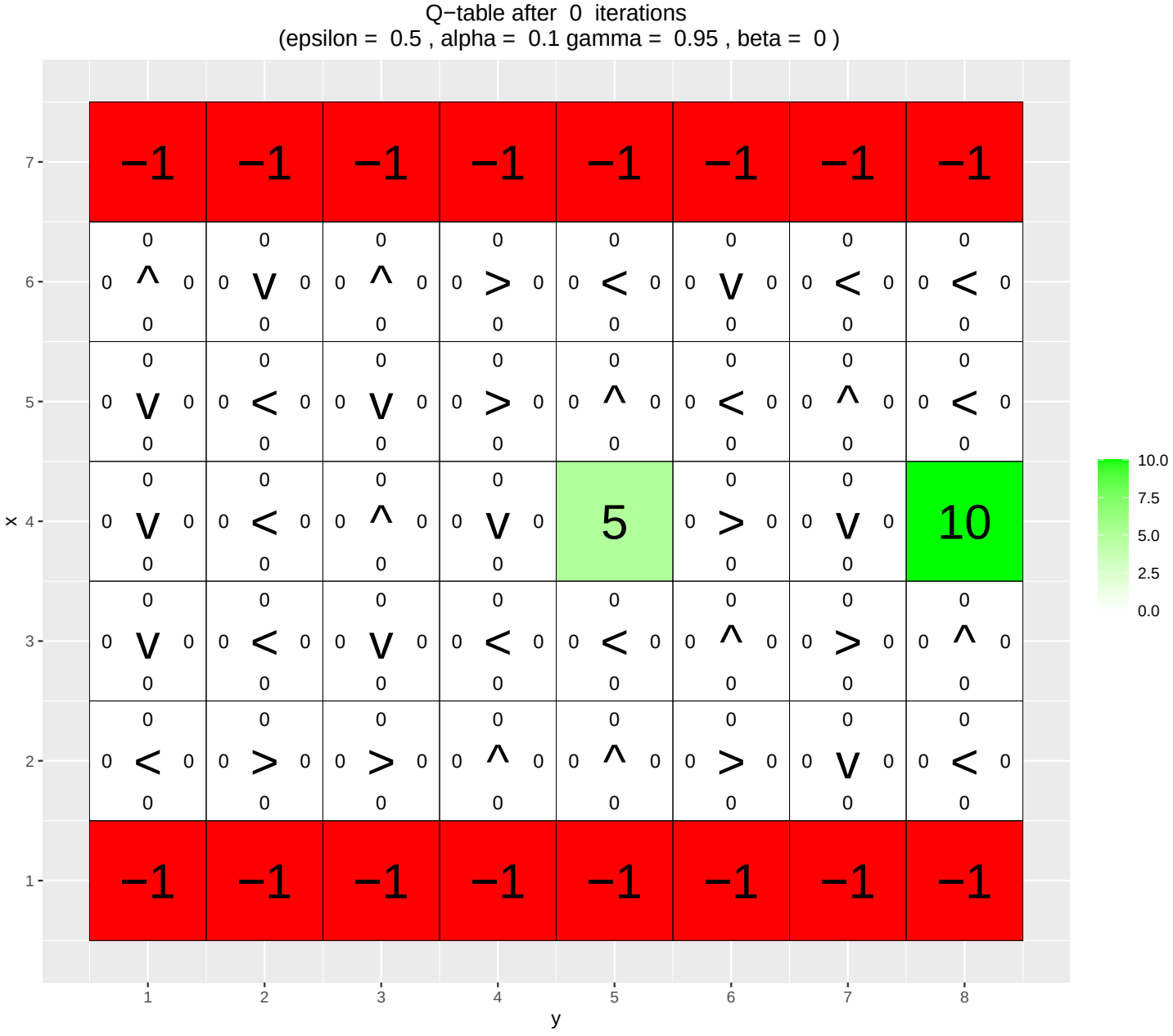
Do the learned Q-values reflect the multiple paths to the positive reward? If not, what could be done?

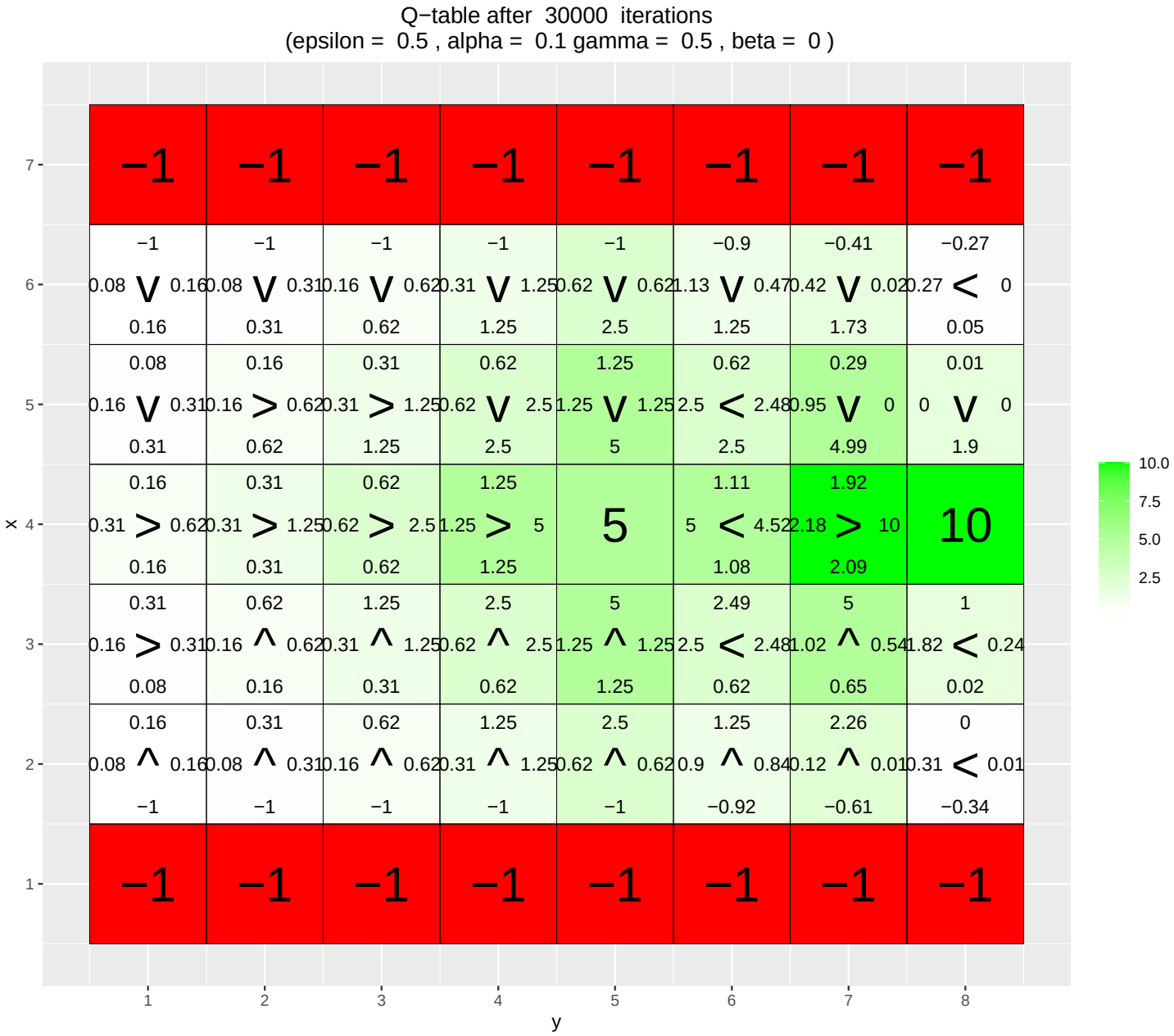
The two routes around the negative-reward states can be learned only if both receive enough exploration. With a fixed start state the agent may favor one route, leaving the alternative not as well estimated. More training, exploration, or changing the starting state would give both routes more coverage.

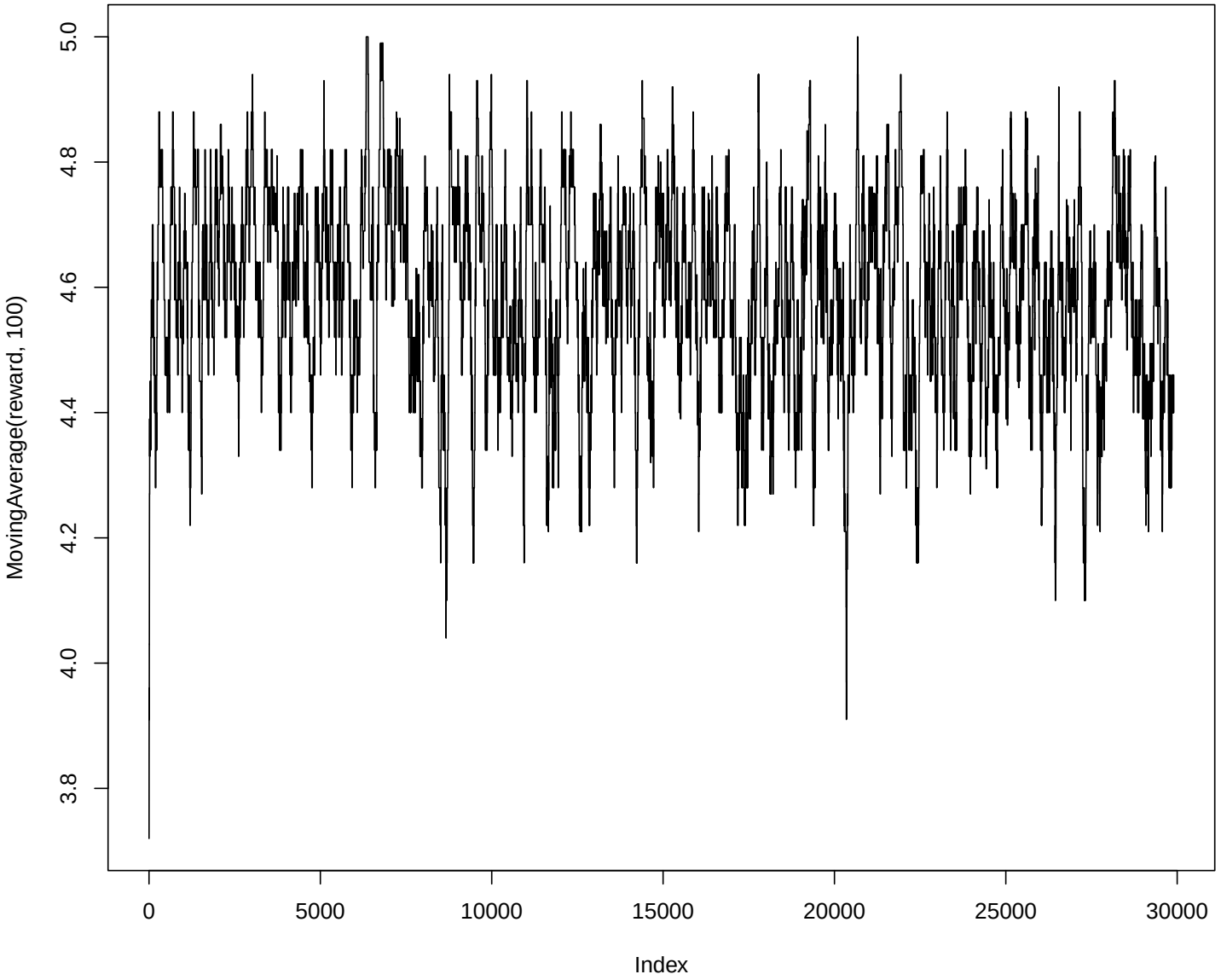
1.2 Environment B: effects of epsilon and gamma

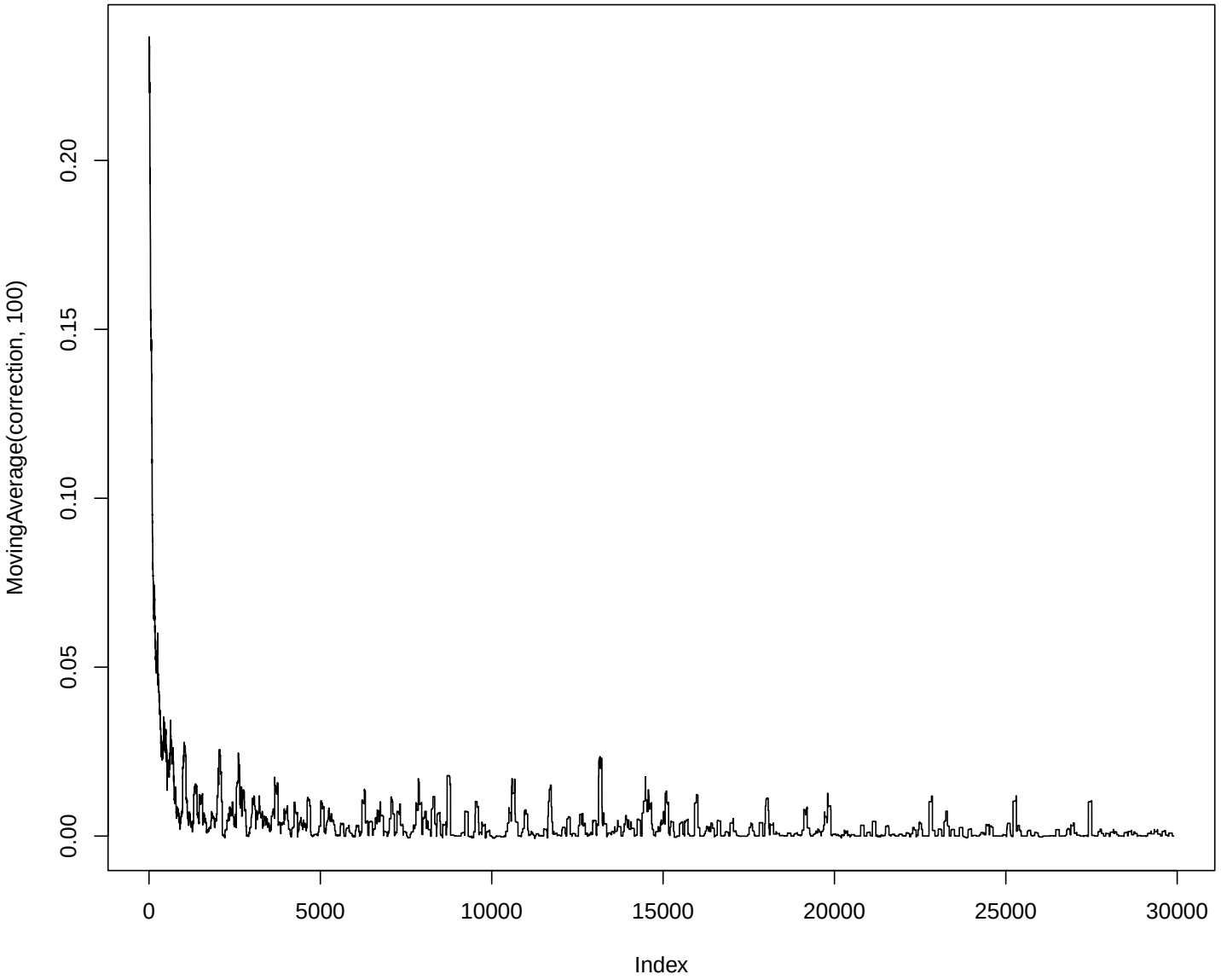
Environment B has negative-reward top and bottom rows, a +5 reward and a +10 reward. The experiment compares gamma values 0.5, 0.75 and 0.95 at epsilon 0.5 and 0.1.

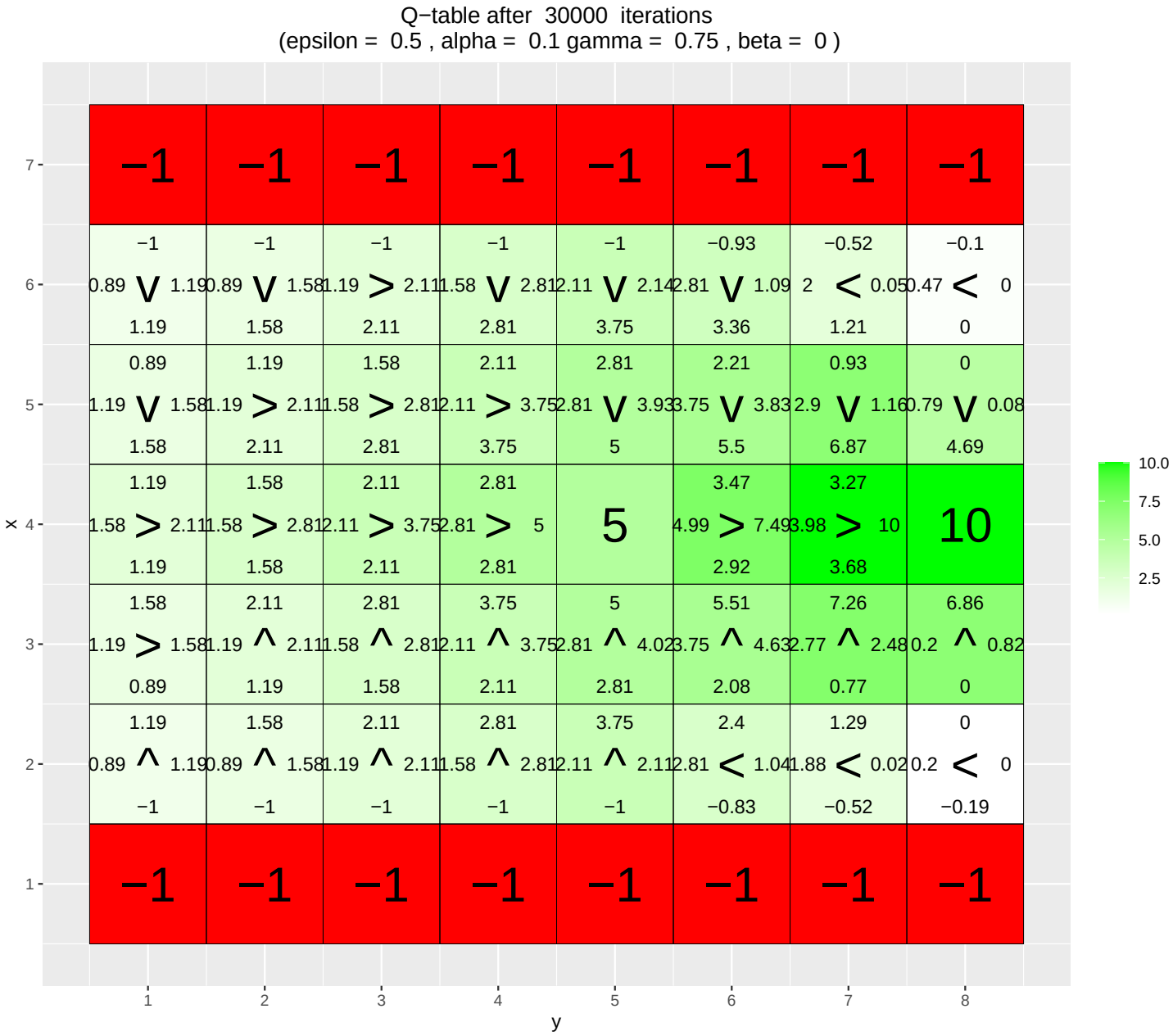
Effects of gamma with epsilon = 0.5

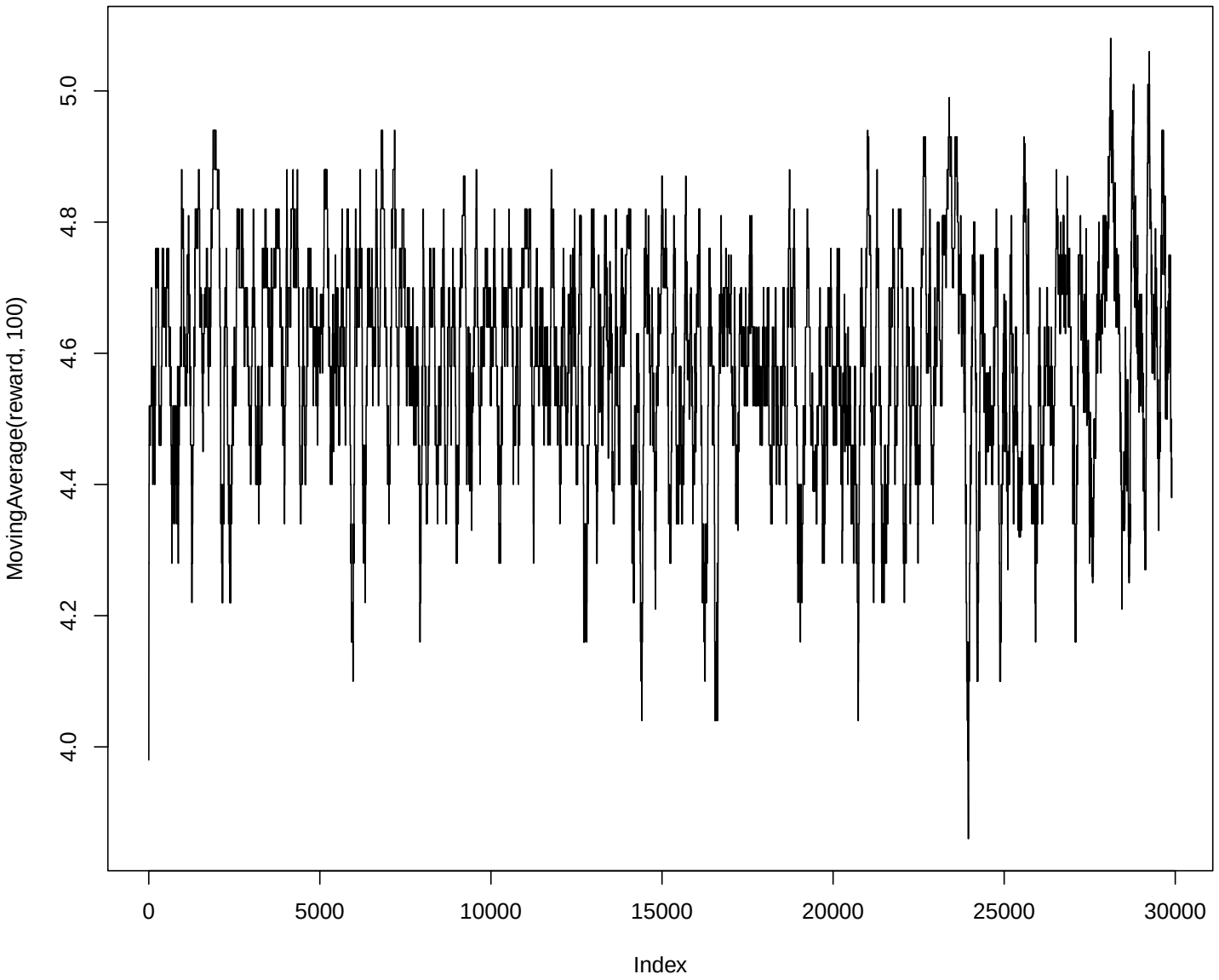


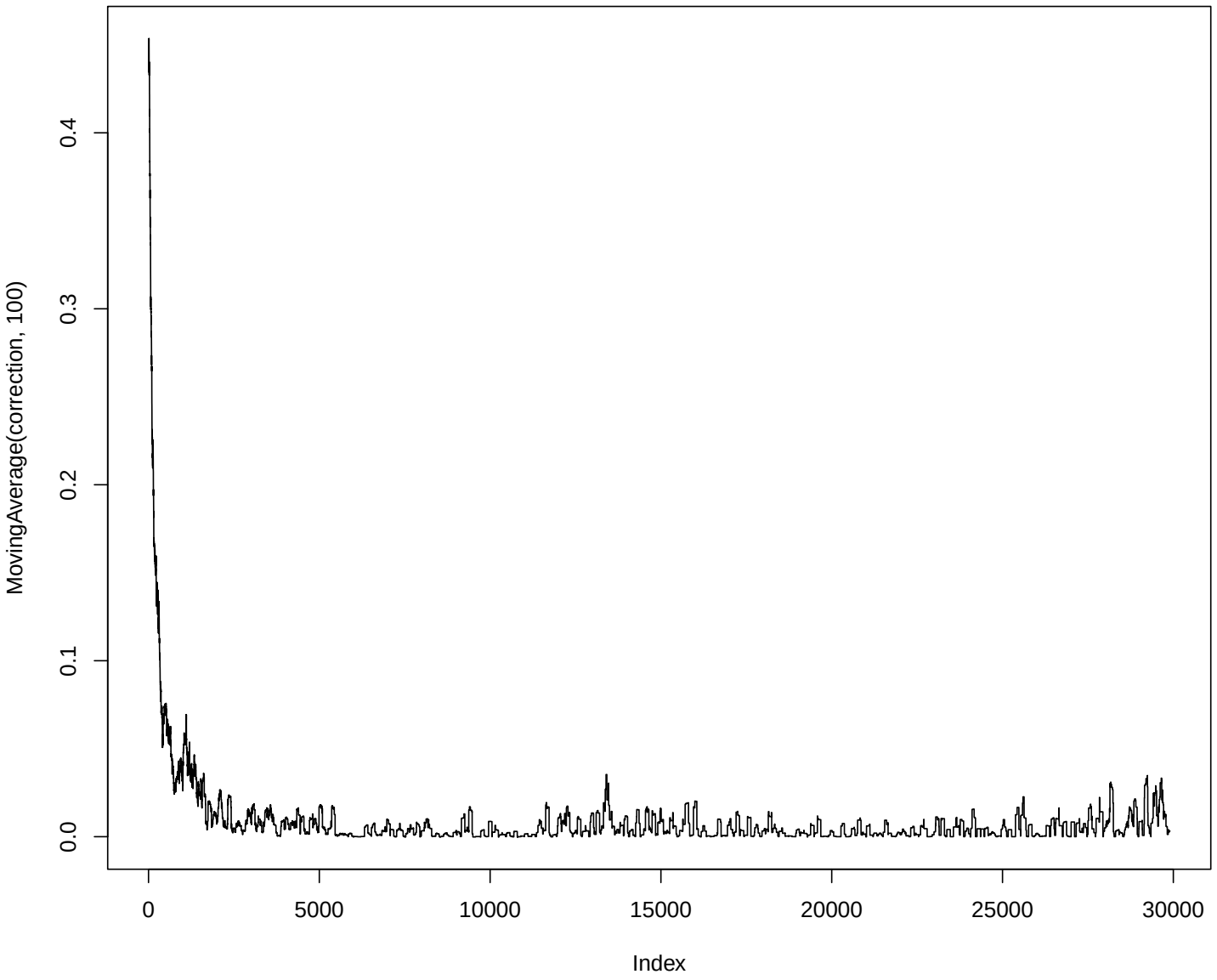

Q−table after 30000 iterations
(epsilon = 0.5 , alpha = 0.1 gamma = 0.5 , beta = 0)
−1 −1 −1 −1 −1 −1 −1 −1
−1 −1 −1 −1 −1 −0.9 −0.41 −0.27
0.08 V 0.16 0.08 V 0.31 0.16 V 0.62 0.31 V 1.25 0.62 V 0.62 1.13 V 0.47 0.42 V 0.02 0.27 < 0
0.16 0.31 0.62 1.25 2.5 1.25 1.73 0.05
0.08 0.16 0.31 0.62 1.25 0.62 0.29 0.01
0.16 V 0.31 0.16 > 0.62 0.31 > 1.25 0.62 V 2.5 1.25 V 1.25 2.5 < 2.48 0.95 V 0 0 V 0
0.31 0.62 1.25 2.5 5 2.5 4.99 1.9
0.16 0.31 0.62 1.25 1.11 1.92
0.31 > 0.62 0.31 > 1.25 0.62 > 2.5 1.25 > 5 5 5 < 4.52 2.18 > 10
0.16 0.31 0.62 1.25 1.08 2.09
0.31 0.62 1.25 2.5 5 2.49 5 1
0.16 > 0.31 0.16 ^ 0.62 0.31 ^ 1.25 0.62 ^ 2.5 1.25 ^ 1.25 2.5 < 2.48 1.02 ^ 0.54 1.82 < 0.24
0.08 0.16 0.31 0.62 1.25 0.62 0.65 0.02
0.16 0.31 0.62 1.25 2.5 1.25 2.26 0
0.08 ^ 0.16 0.08 ^ 0.31 0.16 ^ 0.62 0.31 ^ 1.25 0.62 ^ 0.62 0.9 ^ 0.84 0.12 ^ 0.01 0.31 < 0.01
−1 −1 −1 −1 −1 −0.92 −0.61 −0.34
−1 −1 −1 −1 −1 −1 −1 −1
10.0
7.5
5.0
2.5
x
7
6
5
4
3
2
1
y
1
2
3
4
5
6
7
8

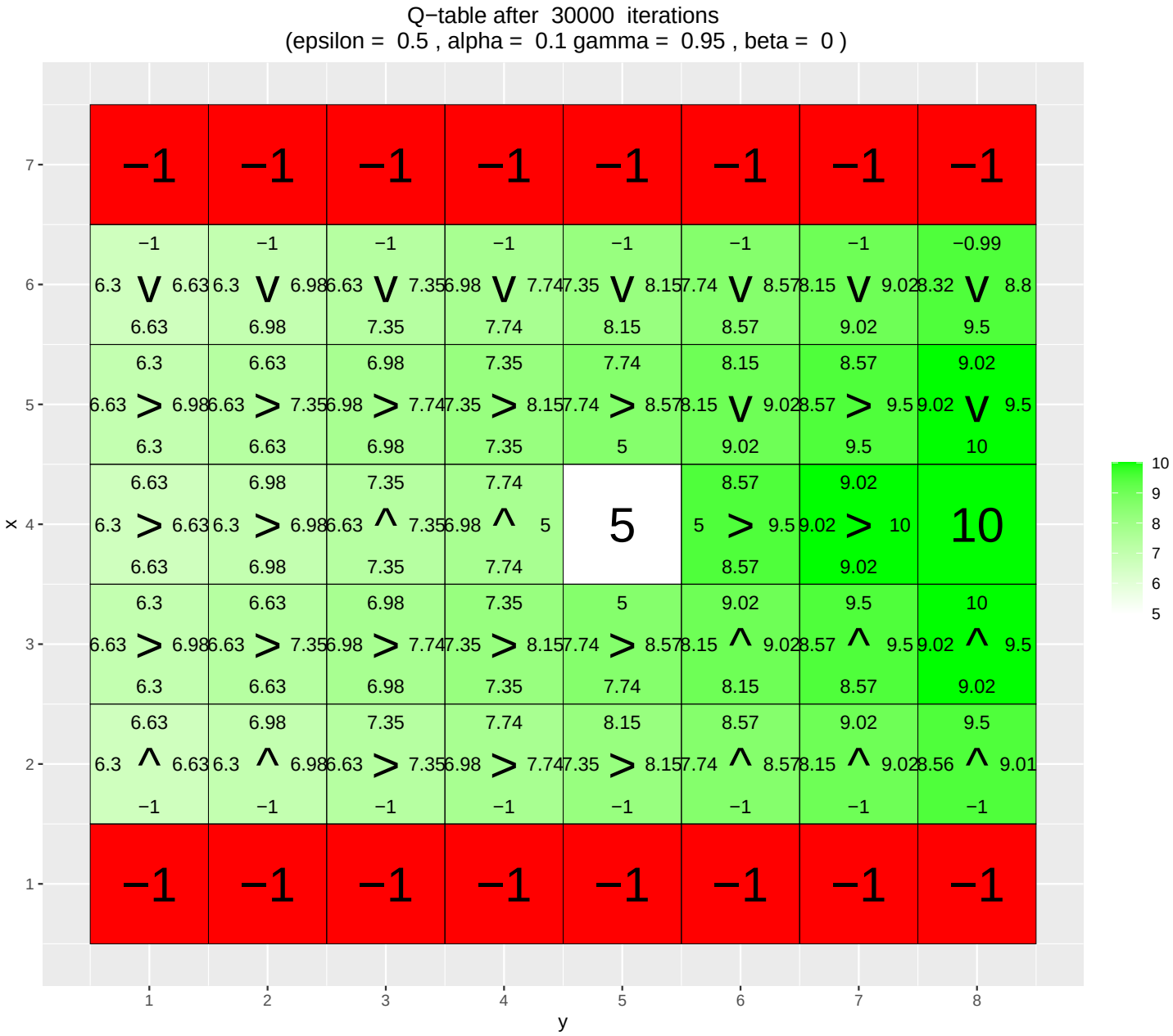

MovingAverage(reward, 100)
5.0
4.8
4.6
4.4
4.2
4.0
3.8
0
5000
10000
15000
20000
25000
30000
Index

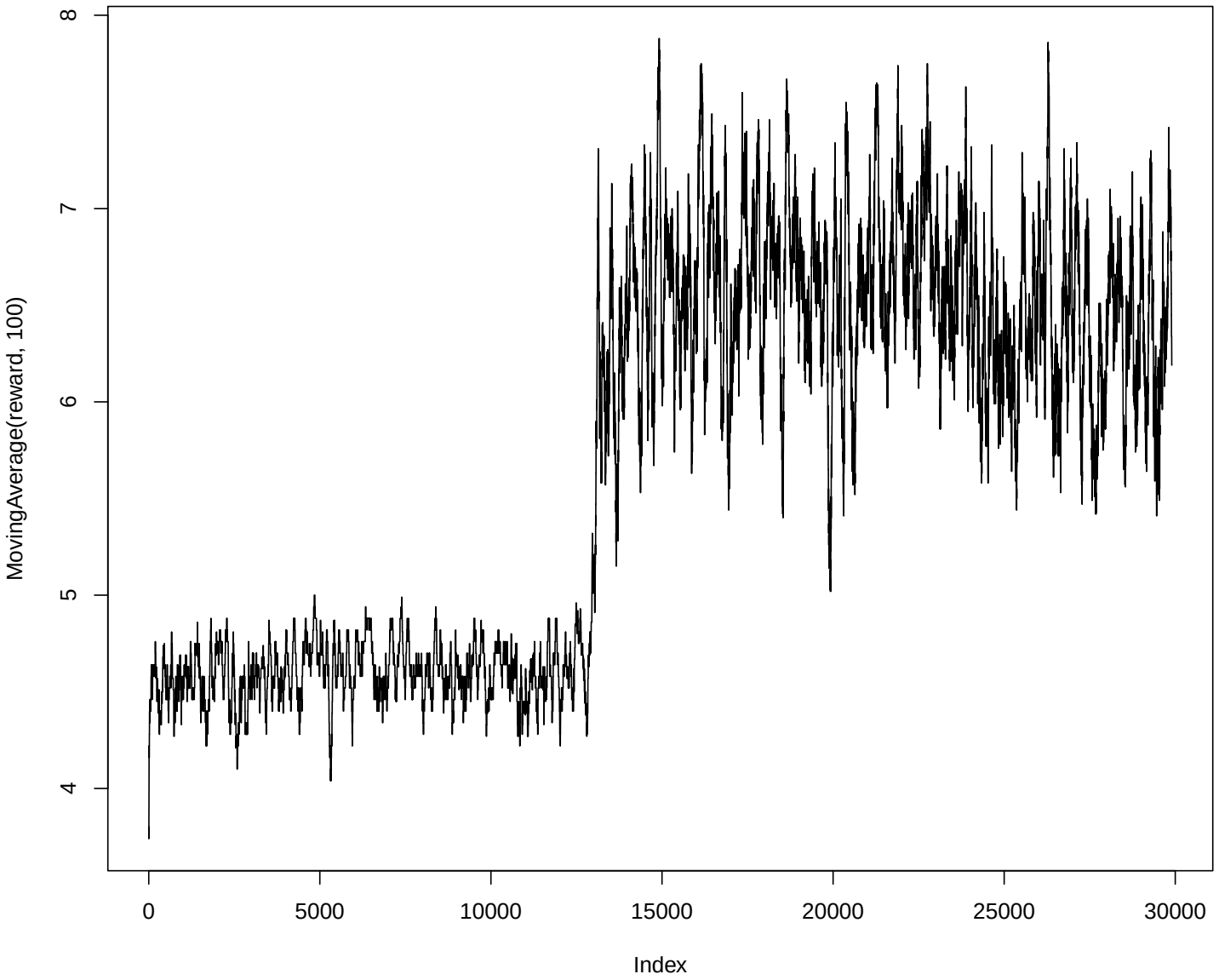

MovingAverage(correction, 100)
0.00
0.05
0.10
0.15
0.20
0
5000
10000
15000
20000
25000
30000
Index

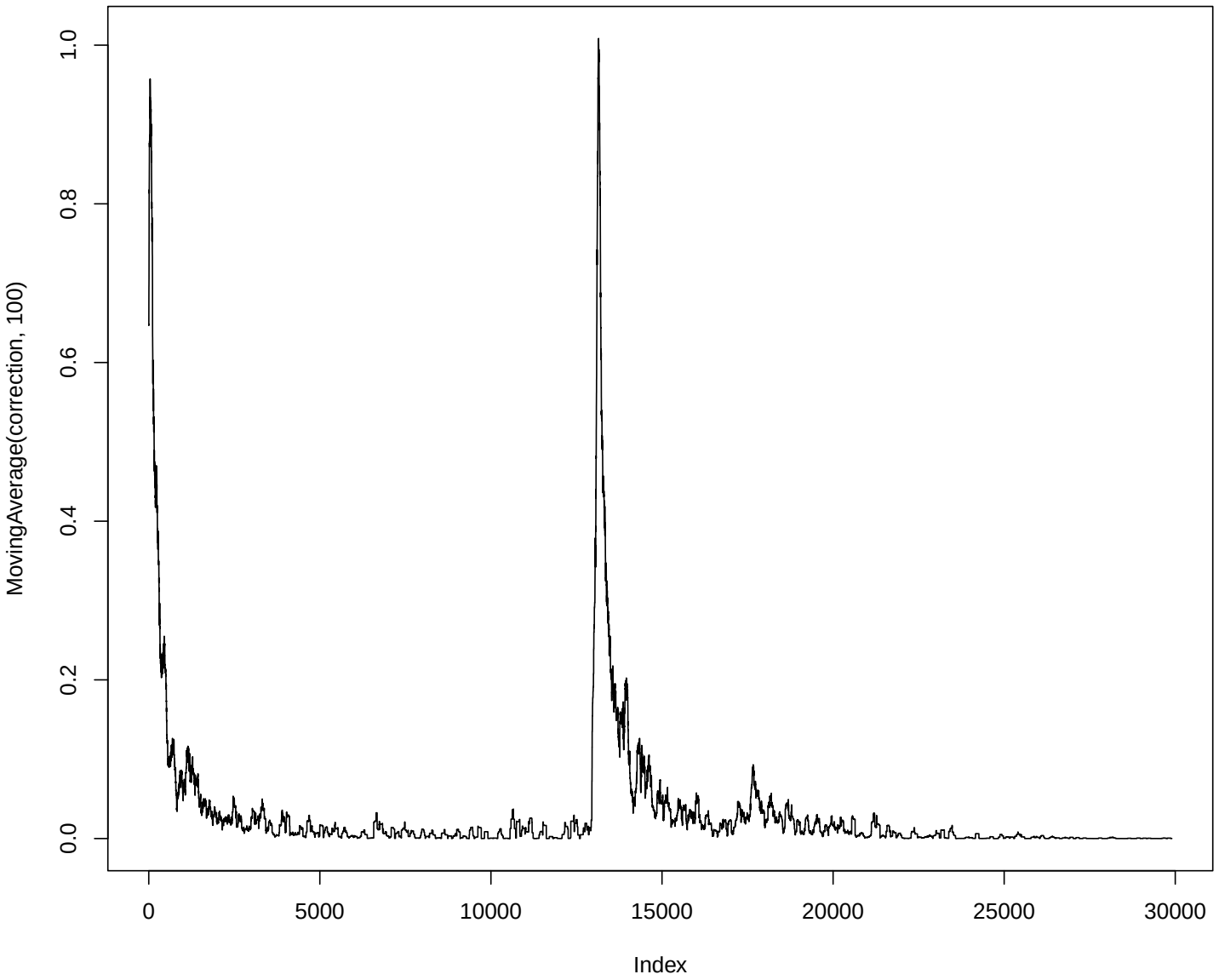

Q−table after 30000 iterations
(epsilon = 0.5 , alpha = 0.1 gamma = 0.75 , beta = 0)
−1 −1 −1 −1 −1 −1 −1 −1
−1 −1 −1 −1 −1 −0.93 −0.52 −0.1
0.89 V 1.19 0.89 V 1.58 1.19 > 2.11 1.58 V 2.81 2.11 V 2.14 2.81 V 1.09 2 < 0.05 0.47 < 0
1.19 1.58 2.11 2.81 3.75 3.36 1.21 0
0.89 1.19 1.58 2.11 2.81 2.21 0.93 0
1.19 V 1.58 1.19 > 2.11 1.58 > 2.81 2.11 > 3.75 2.81 V 3.93 3.75 V 3.83 2.9 V 1.16 0.79 V 0.08
1.58 2.11 2.81 3.75 5 5.5 6.87 4.69
1.19 1.58 2.11 2.81 3.47 3.27
1.58 > 2.11 1.58 > 2.81 2.11 > 3.75 2.81 > 5 5 4.99 > 7.49 3.98 > 10 10
1.19 1.58 2.11 2.81 2.92 3.68
1.58 2.11 2.81 3.75 5 5.51 7.26 6.86
1.19 > 1.58 1.19 ^ 2.11 1.58 ^ 2.81 2.11 ^ 3.75 2.81 ^ 4.02 3.75 ^ 4.63 2.77 ^ 2.48 0.2 ^ 0.82
0.89 1.19 1.58 2.11 2.81 2.08 0.77 0
1.19 1.58 2.11 2.81 3.75 2.4 1.29 0
0.89 ^ 1.19 0.89 ^ 1.58 1.19 ^ 2.11 1.58 ^ 2.81 2.11 ^ 2.11 2.81 < 1.04 1.88 < 0.02 0.2 < 0
−1 −1 −1 −1 −1 −0.83 −0.52 −0.19
−1 −1 −1 −1 −1 −1 −1 −1
x
y
1 2 3 4 5 6 7 8
1 2 3 4 5 6 7
10.0
7.5
5.0
2.5


MovingAverage(reward, 100)
4.0
4.2
4.4
4.6
4.8
5.0
0
5000
10000
15000
20000
25000
30000
Index

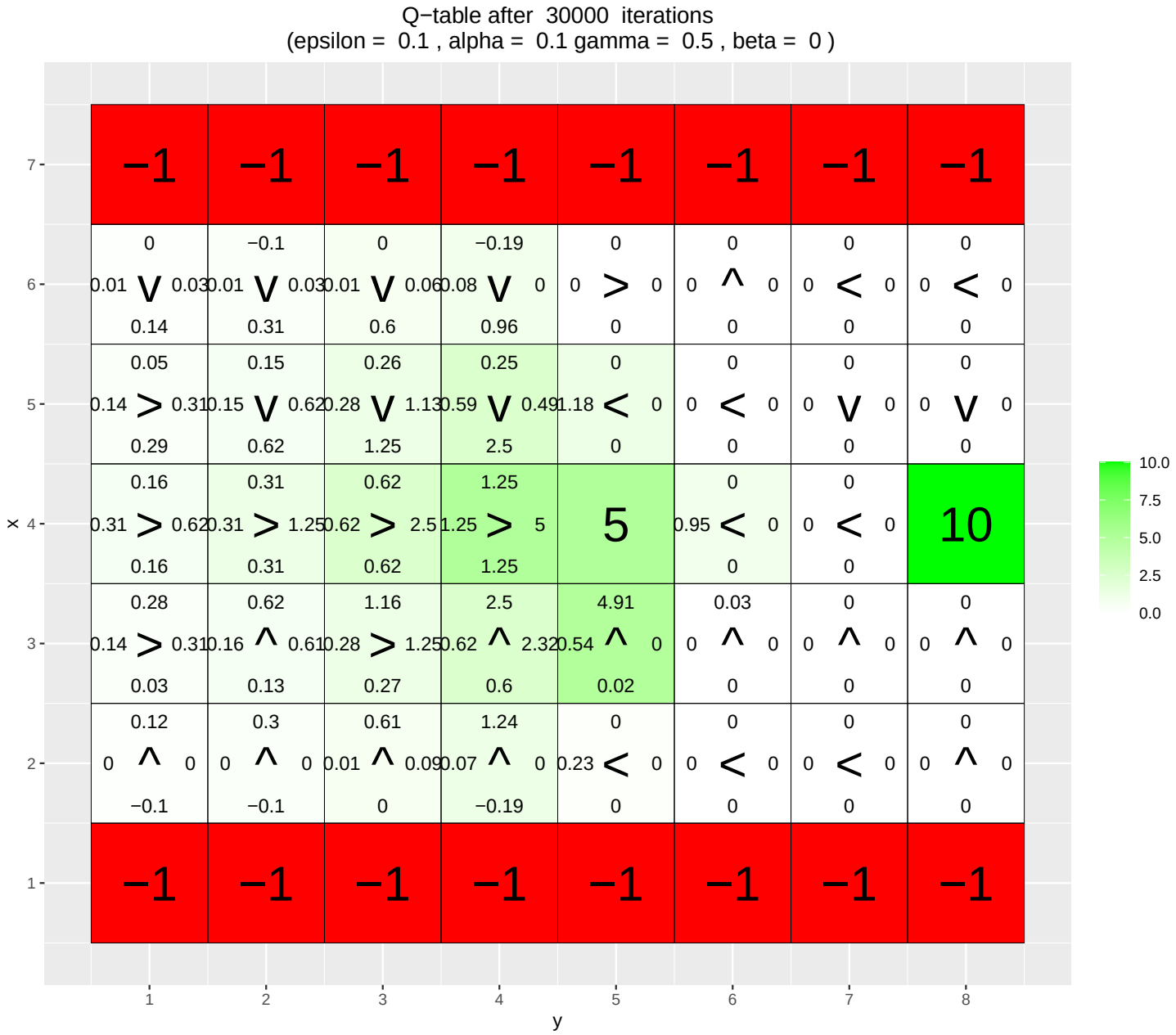

MovingAverage(correction, 100)
0.0
0.1
0.2
0.3
0.4
0
5000
10000
15000
20000
25000
30000
Index

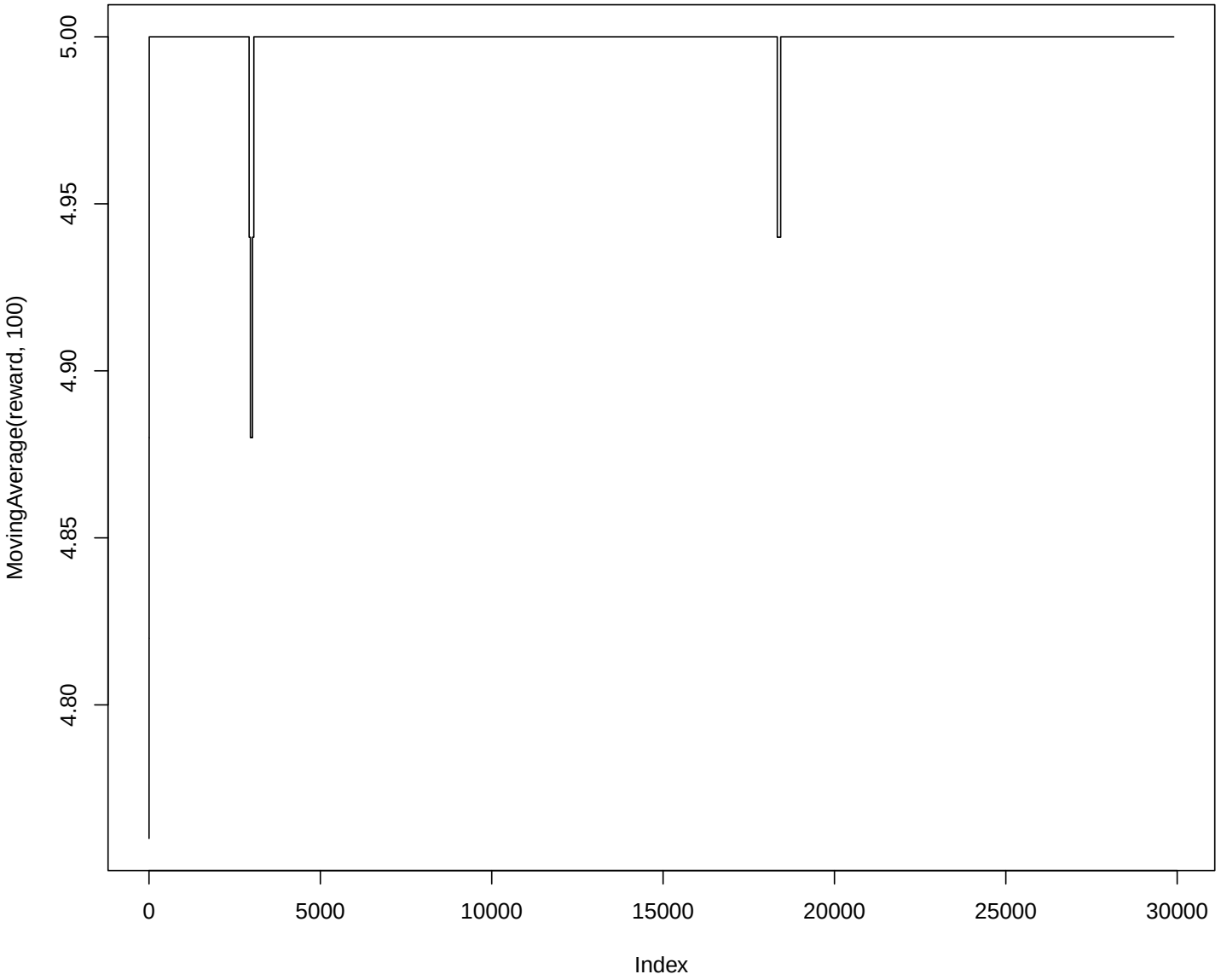

Q−table after 30000 iterations
(epsilon = 0.5 , alpha = 0.1 gamma = 0.95 , beta = 0)
−1 −1 −1 −1 −1 −1 −1 −1
−1 −1 −1 −1 −1 −1 −1 −0.99
6.3 V 6.63 6.3 V 6.98 6.63 V 7.35 6.98 V 7.74 7.35 V 8.15 7.74 V 8.57 8.15 V 9.02 8.32 V 8.8
6.63 6.98 7.35 7.74 8.15 8.57 9.02 9.5
6.3 6.63 6.98 7.35 7.74 8.15 8.57 9.02
6.63 > 6.98 6.63 > 7.35 6.98 > 7.74 7.35 > 8.15 7.74 > 8.57 8.15 V 9.02 8.57 > 9.5 9.02 V 9.5
6.3 6.63 6.98 7.35 5 9.02 9.5 10
6.63 6.98 7.35 7.74 8.57 9.02
6.3 > 6.63 6.3 > 6.98 6.63 ^ 7.35 6.98 ^ 5 5 > 9.5 9.02 > 10 10
6.63 6.98 7.35 7.74 8.57 9.02
6.3 6.63 6.98 7.35 5 9.02 9.5 10
6.63 > 6.98 6.63 > 7.35 6.98 > 7.74 7.35 > 8.15 7.74 > 8.57 8.15 ^ 9.02 8.57 ^ 9.5 9.02 ^ 9.5
6.3 6.63 6.98 7.35 7.74 8.15 8.57 9.02
6.63 6.98 7.35 7.74 8.15 8.57 9.02 9.5
6.3 ^ 6.63 6.3 ^ 6.98 6.63 > 7.35 6.98 > 7.74 7.35 > 8.15 7.74 ^ 8.57 8.15 ^ 9.02 8.56 ^ 9.01
−1 −1 −1 −1 −1 −1 −1 −1
−1 −1 −1 −1 −1 −1 −1 −1
x
y
1 2 3 4 5 6 7 8
1 2 3 4 5 6 7
10 9 8 7 6 5

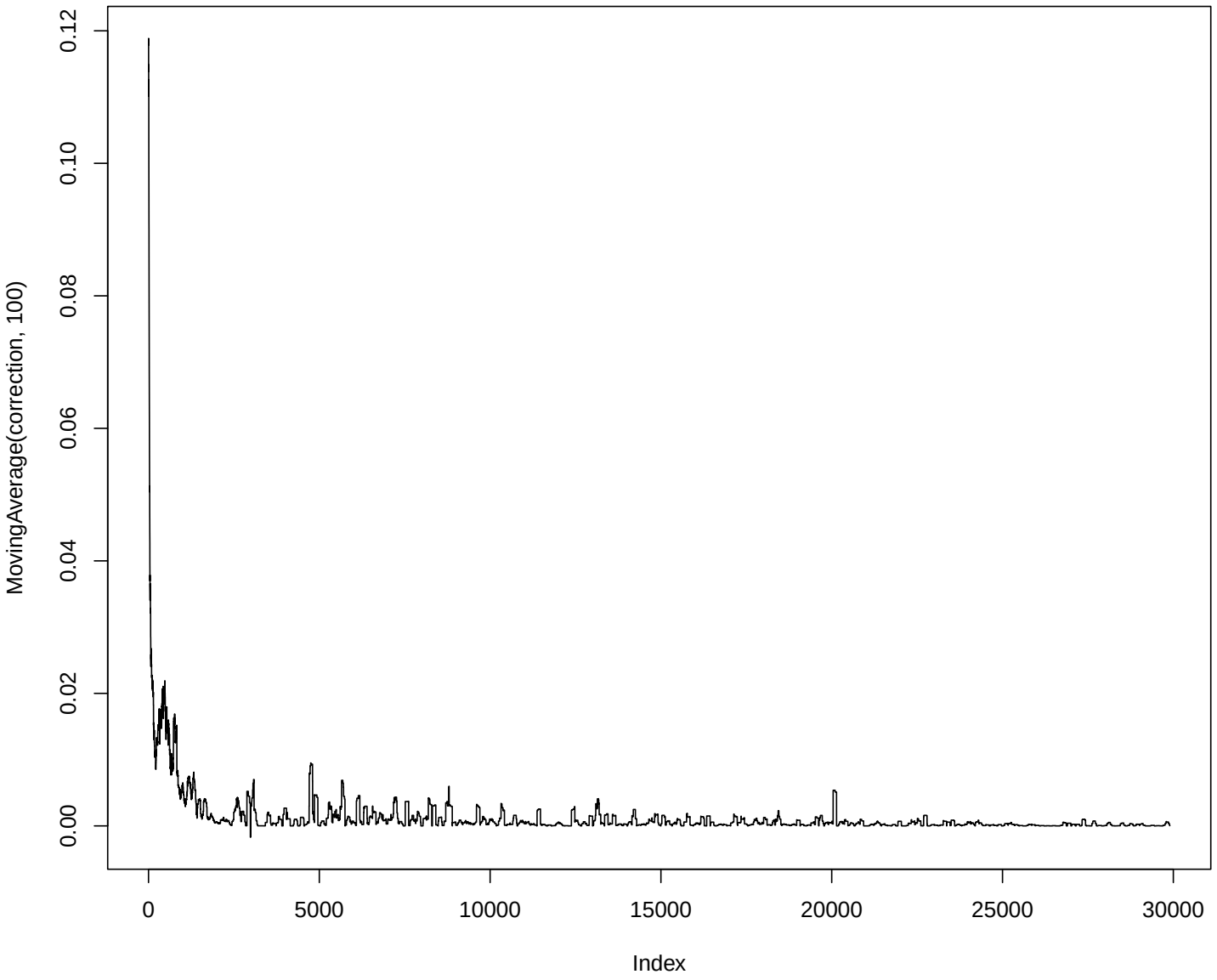

MovingAverage(reward, 100)
4
5
6
7
8
0
5000
10000
15000
20000
25000
30000
Index

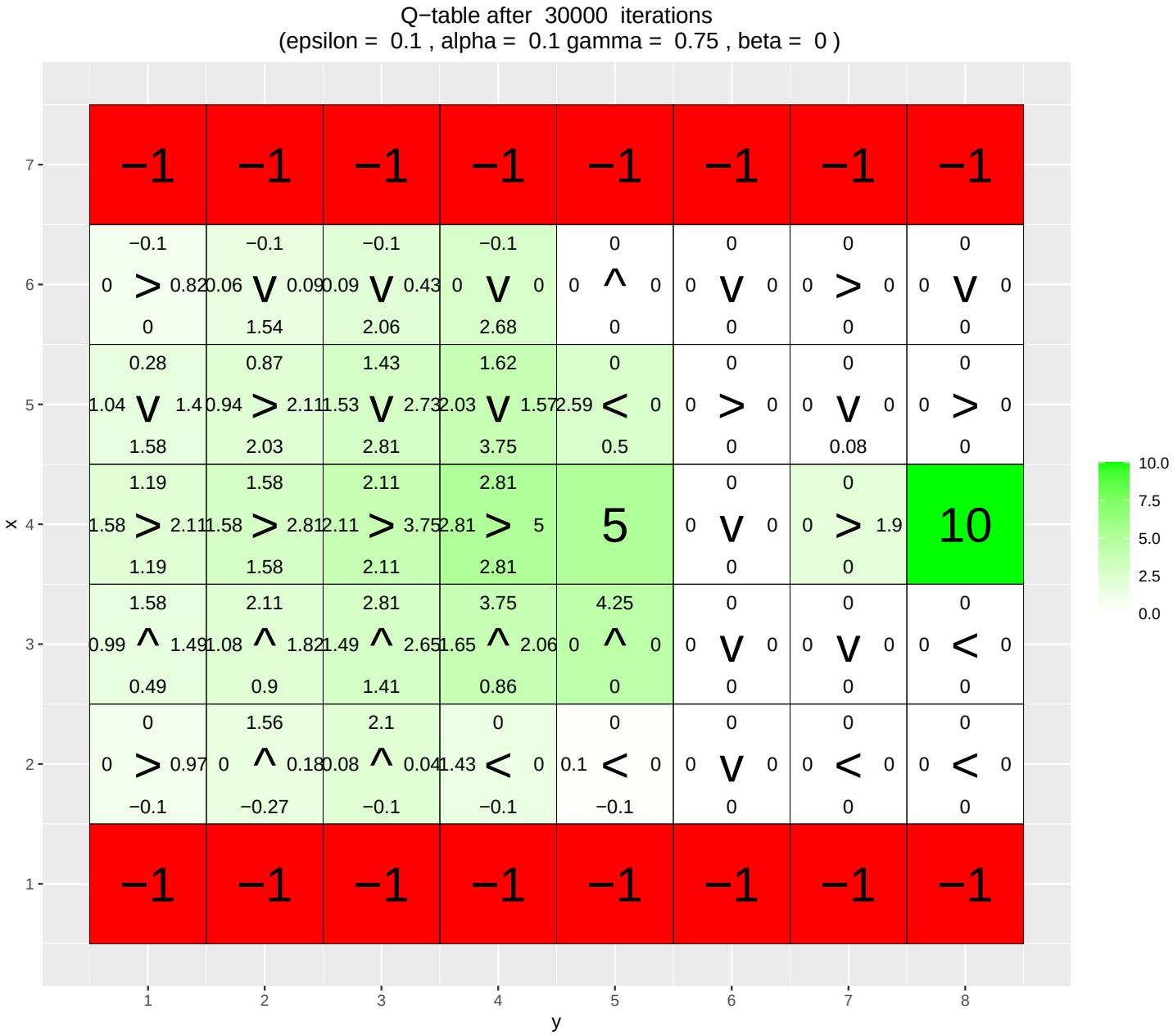


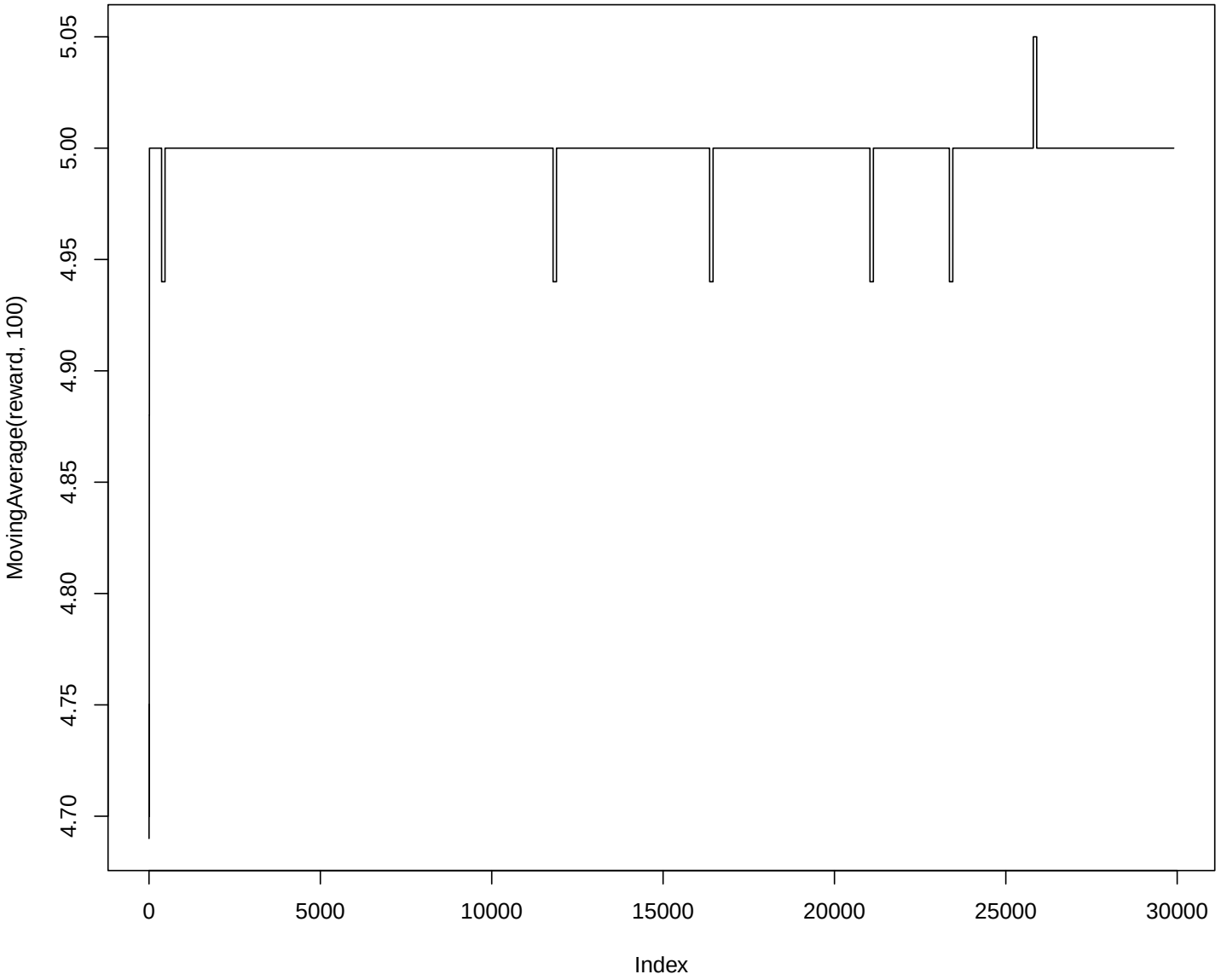
Increasing gamma makes later rewards more influential, so the agent is more willing to take a longer route to the +10 reward instead of stopping at the nearer +5 reward. With gamma = 0.5 and 0.75, we can see that the nearer 5 reward is preferred. With gamma 0.95 makes the farther route 10 worth going to.

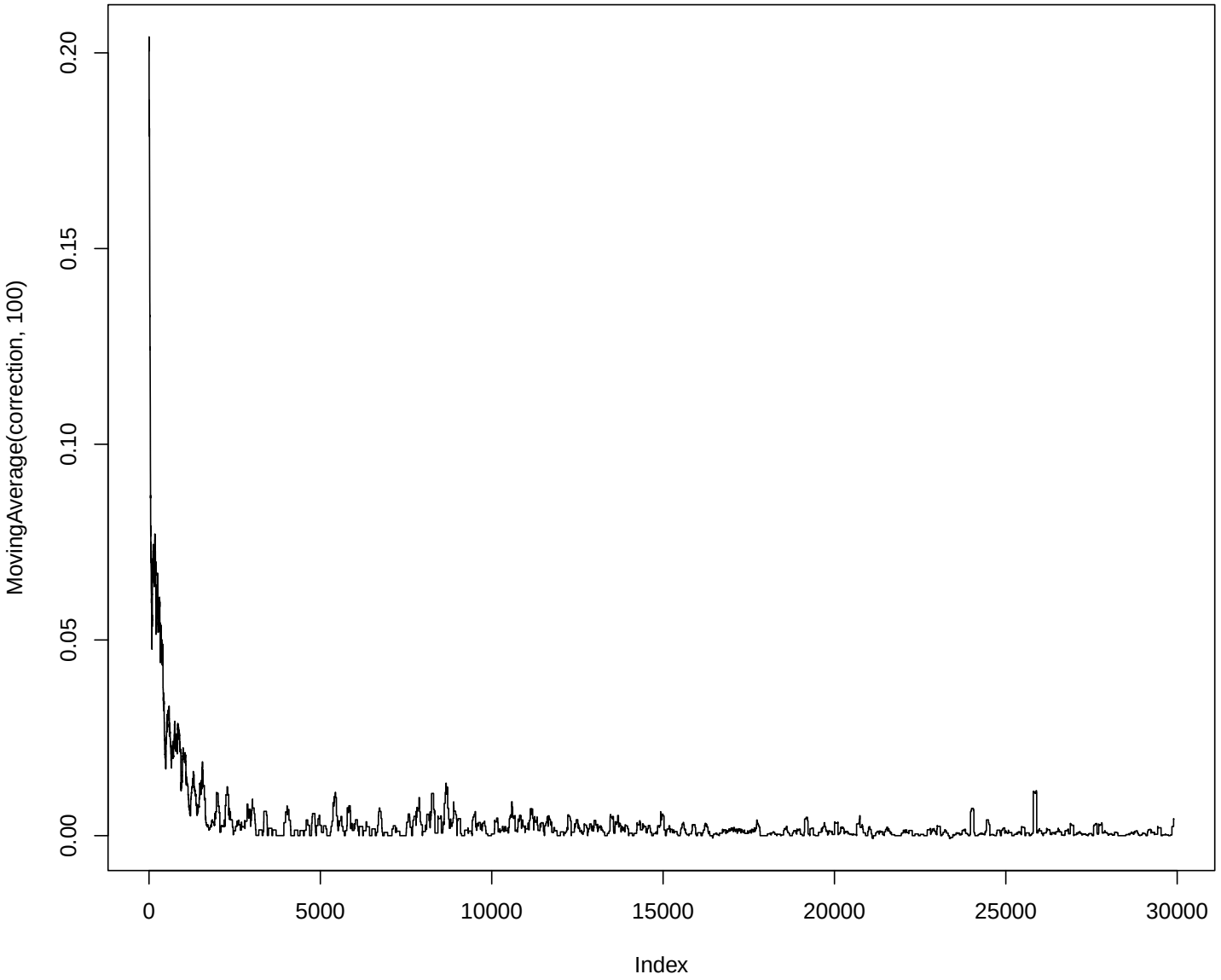

Q−table after 30000 iterations
(epsilon = 0.1 , alpha = 0.1 gamma = 0.5 , beta = 0)
−1 −1 −1 −1 −1 −1 −1 −1
0 −0.1 0 −0.19 0 0 0 0
0.01 V 0.03 0.01 V 0.03 0.01 V 0.06 0.08 V 0 0 > 0 0 ^ 0 0 < 0 0 < 0
0.14 0.31 0.6 0.96 0 0 0 0
0.05 0.15 0.26 0.25 0 0 0 0
0.14 > 0.31 0.15 V 0.62 0.28 V 1.13 0.59 V 0.49 1.18 < 0 0 < 0 0 V 0 0 V 0
0.29 0.62 1.25 2.5 0 0 0 0
0.16 0.31 0.62 1.25 0 0 0
0.31 > 0.62 0.31 > 1.25 0.62 > 2.5 1.25 > 5 5 0.95 < 0 0 < 0 10
0.16 0.31 0.62 1.25 0 0 0
0.28 0.62 1.16 2.5 4.91 0.03 0 0
0.14 > 0.31 0.16 ^ 0.61 0.28 > 1.25 0.62 ^ 2.32 0.54 ^ 0 0 ^ 0 0 ^ 0 0 ^ 0
0.03 0.13 0.27 0.6 0.02 0 0 0
0.12 0.3 0.61 1.24 0 0 0 0
0 ^ 0 0 ^ 0 0.01 ^ 0.09 0.07 ^ 0 0.23 < 0 0 < 0 0 < 0 0 ^ 0
−0.1 −0.1 0 −0.19 0 0 0 0
−1 −1 −1 −1 −1 −1 −1 −1
x
y
1 2 3 4 5 6 7 8
1 2 3 4 5 6 7
10.0
7.5
5.0
2.5
0.0

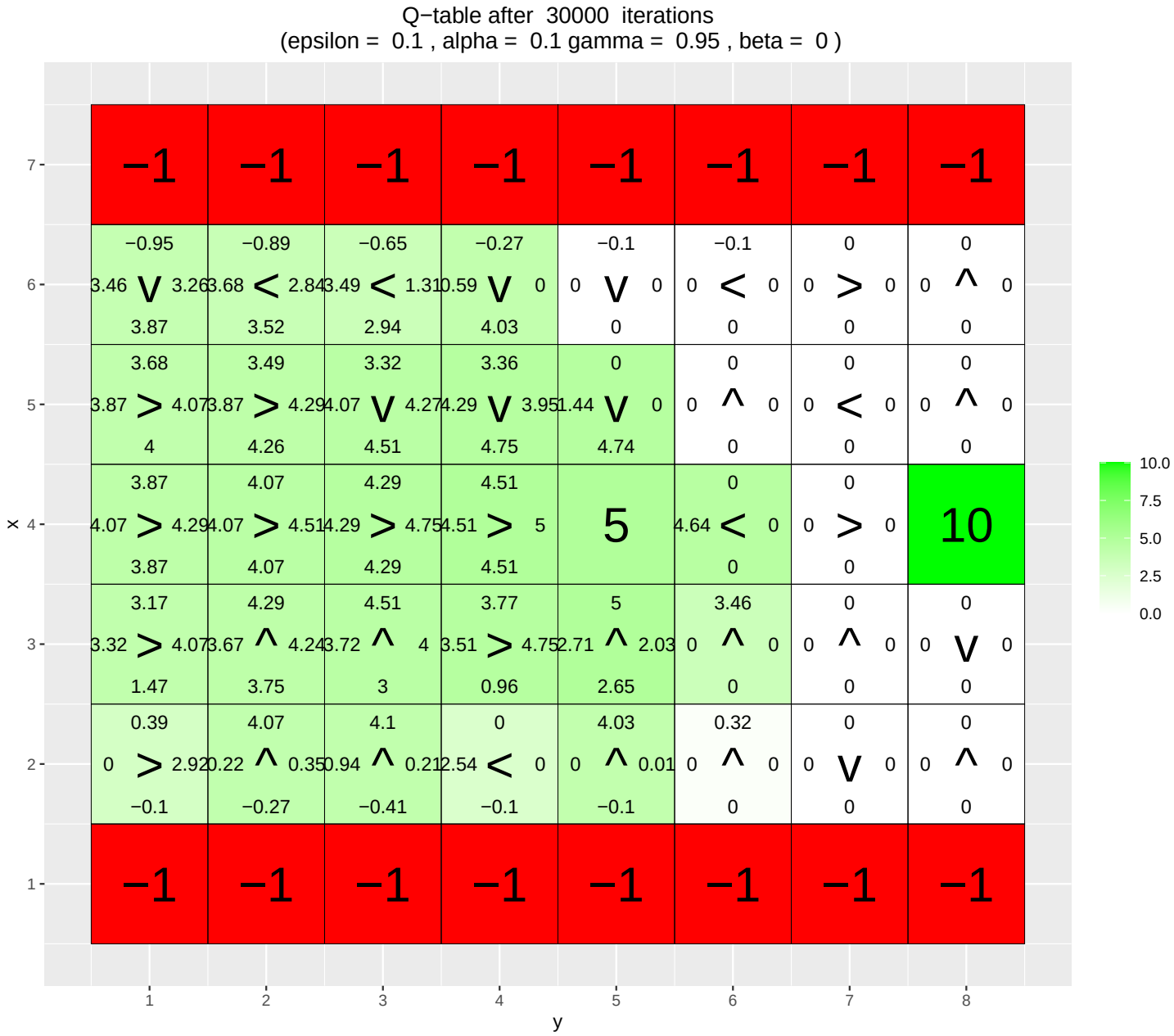

MovingAverage(reward, 100)
5.00
4.95
4.90
4.85
4.80
0
5000
10000
15000
20000
25000
30000
Index

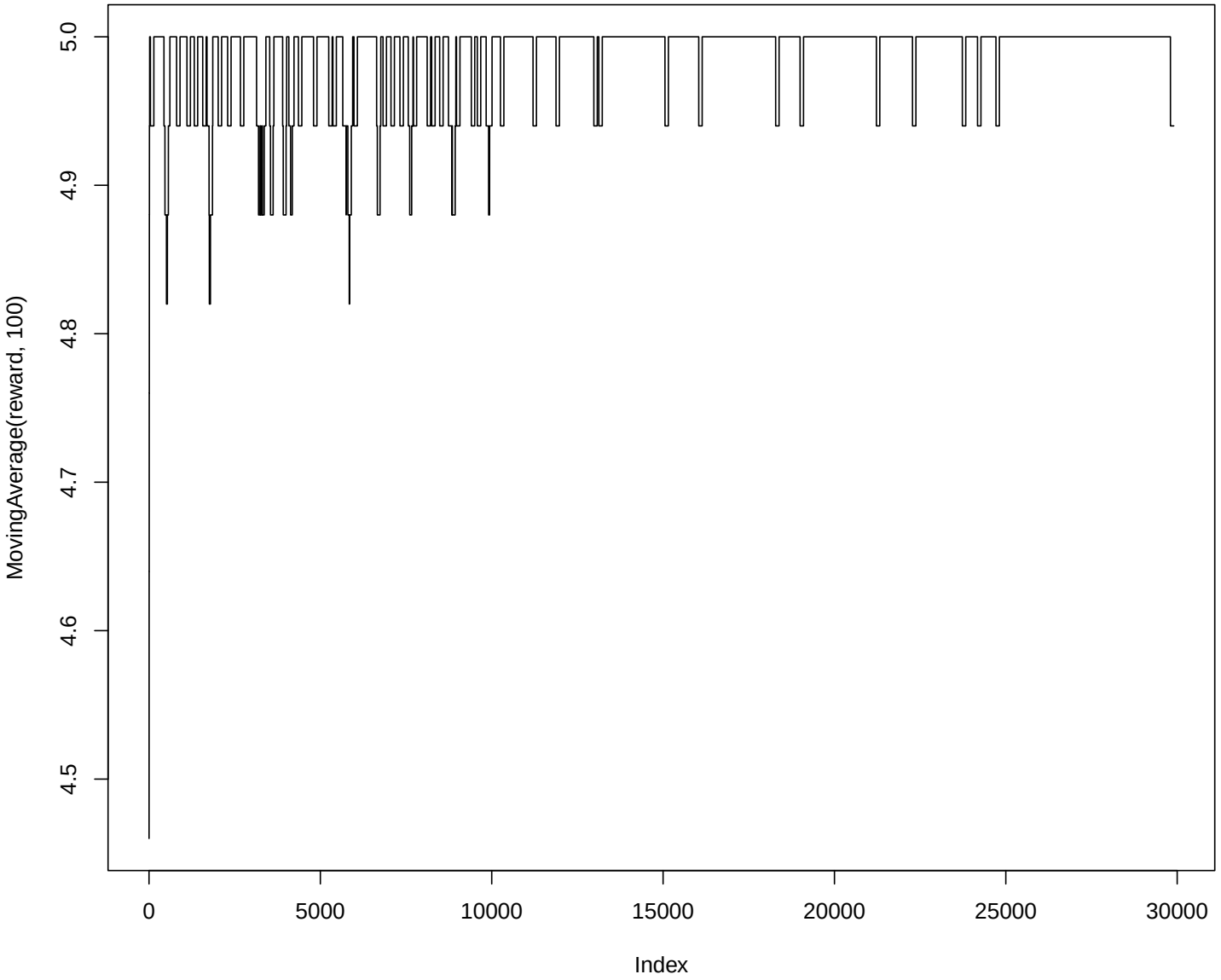

MovingAverage(correction, 100)
0.00
0.02
0.04
0.06
0.08
0.10
0.12
0
5000
10000
15000
20000
25000
30000
Index

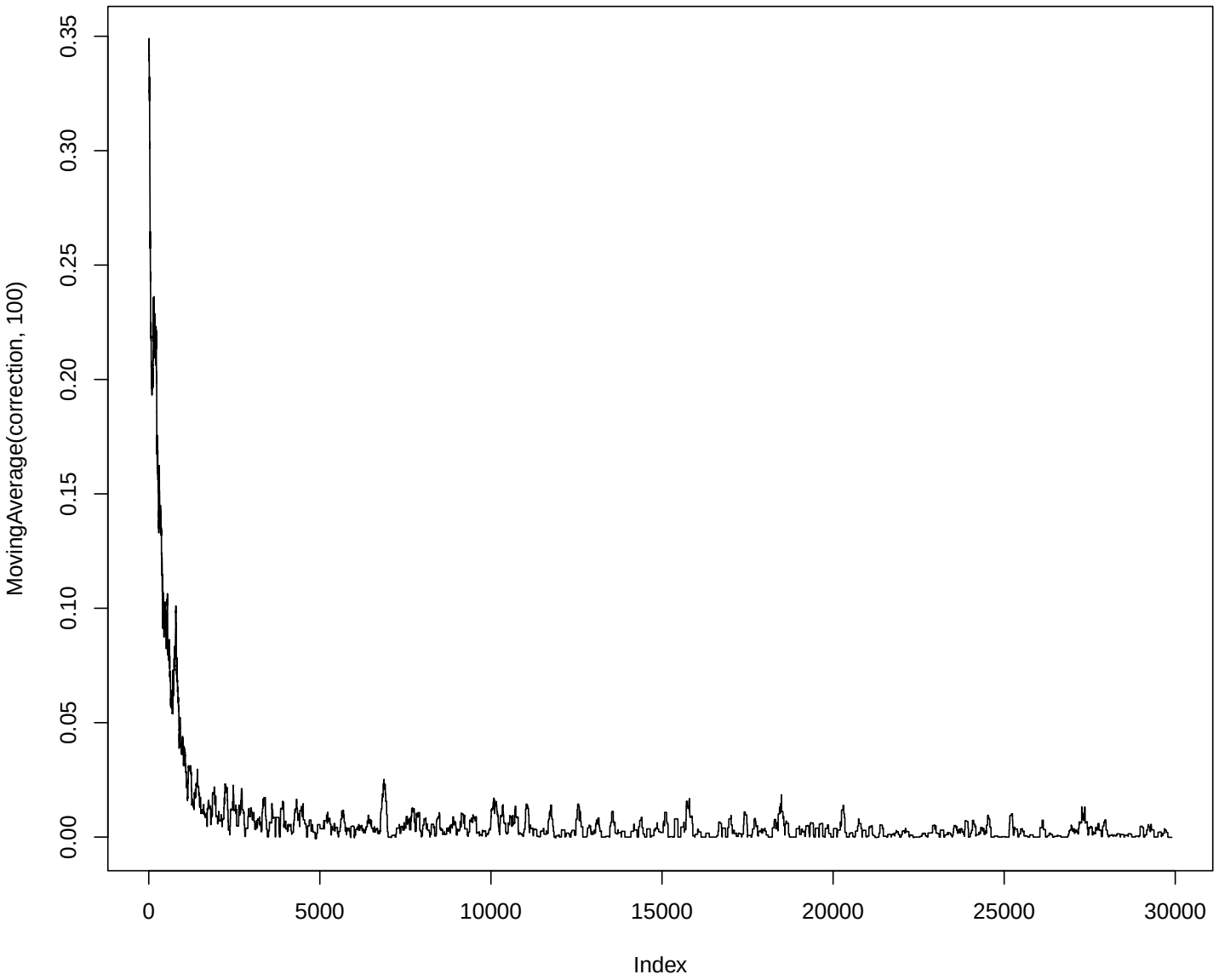

Q−table after 30000 iterations
(epsilon = 0.1 , alpha = 0.1 gamma = 0.75 , beta = 0)
−1 −1 −1 −1 −1 −1 −1 −1
−0.1 −0.1 −0.1 −0.1 0 0 0 0
0 > 0.82 0.06 v 0.09 0.09 v 0.43 0 v 0 0 ^ 0 0 v 0 0 > 0 0 v 0
0 1.54 2.06 2.68 0 0 0 0
0.28 0.87 1.43 1.62 0 0 0 0
1.04 v 1.4 0.94 > 2.11 1.53 v 2.73 2.03 v 1.57 2.59 < 0 0 > 0 0 v 0 0 > 0
1.58 2.03 2.81 3.75 0.5 0 0.08 0
1.19 1.58 2.11 2.81 0 0
1.58 > 2.11 1.58 > 2.81 2.11 > 3.75 2.81 > 5 5 0 v 0 0 > 1.9 10
1.19 1.58 2.11 2.81 0 0
1.58 2.11 2.81 3.75 4.25 0 0 0
0.99 ^ 1.49 1.08 ^ 1.82 1.49 ^ 2.65 1.65 ^ 2.06 0 ^ 0 0 v 0 0 v 0 0 < 0
0.49 0.9 1.41 0.86 0 0 0 0
0 1.56 2.1 0 0 0 0 0
0 > 0.97 0 ^ 0.18 0.08 ^ 0.04 1.43 < 0 0.1 < 0 0 v 0 0 < 0 0 < 0
−0.1 −0.27 −0.1 −0.1 −0.1 0 0 0
−1 −1 −1 −1 −1 −1 −1 −1
x
y
1 2 3 4 5 6 7 8
1 2 3 4 5 6 7
10.0 7.5 5.0 2.5 0.0


MovingAverage(reward, 100)
5.05
5.00
4.95
4.90
4.85
4.80
4.75
4.70
0
5000
10000
15000
20000
25000
30000
Index


MovingAverage(correction, 100)
0.00
0.05
0.10
0.15
0.20
0
5000
10000
15000
20000
25000
30000
Index


Q−table after 30000 iterations
(epsilon = 0.1 , alpha = 0.1 gamma = 0.95 , beta = 0)
−1 −1 −1 −1 −1 −1 −1 −1
−0.95 −0.89 −0.65 −0.27 −0.1 −0.1 0 0
3.46 V 3.26 3.68 < 2.84 3.49 < 1.31 0.59 V 0 0 V 0 0 < 0 0 > 0 0 ^ 0
3.87 3.52 2.94 4.03 0 0 0 0
3.68 3.49 3.32 3.36 0 0 0 0
3.87 > 4.07 3.87 > 4.29 4.07 V 4.27 4.29 V 3.95 1.44 V 0 0 ^ 0 0 < 0 0 ^ 0
4 4.26 4.51 4.75 4.74 0 0 0
3.87 4.07 4.29 4.51 0 0 0
4.07 > 4.29 4.07 > 4.51 4.29 > 4.75 4.51 > 5 5 4.64 < 0 0 > 0 10
3.87 4.07 4.29 4.51 0 0 0
3.17 4.29 4.51 3.77 5 3.46 0 0
3.32 > 4.07 3.67 ^ 4.24 3.72 ^ 4 3.51 > 4.75 2.71 ^ 2.03 0 ^ 0 0 ^ 0 0 V 0
1.47 3.75 3 0.96 2.65 0 0 0
0.39 4.07 4.1 0 4.03 0.32 0 0
0 > 2.92 0.22 ^ 0.35 0.94 ^ 0.21 2.54 < 0 0 ^ 0.01 0 ^ 0 0 V 0 0 ^ 0
−0.1 −0.27 −0.41 −0.1 −0.1 0 0 0
−1 −1 −1 −1 −1 −1 −1 −1
x
7 6 5 4 3 2 1
1 2 3 4 5 6 7 8
y
10.0 7.5 5.0 2.5 0.0

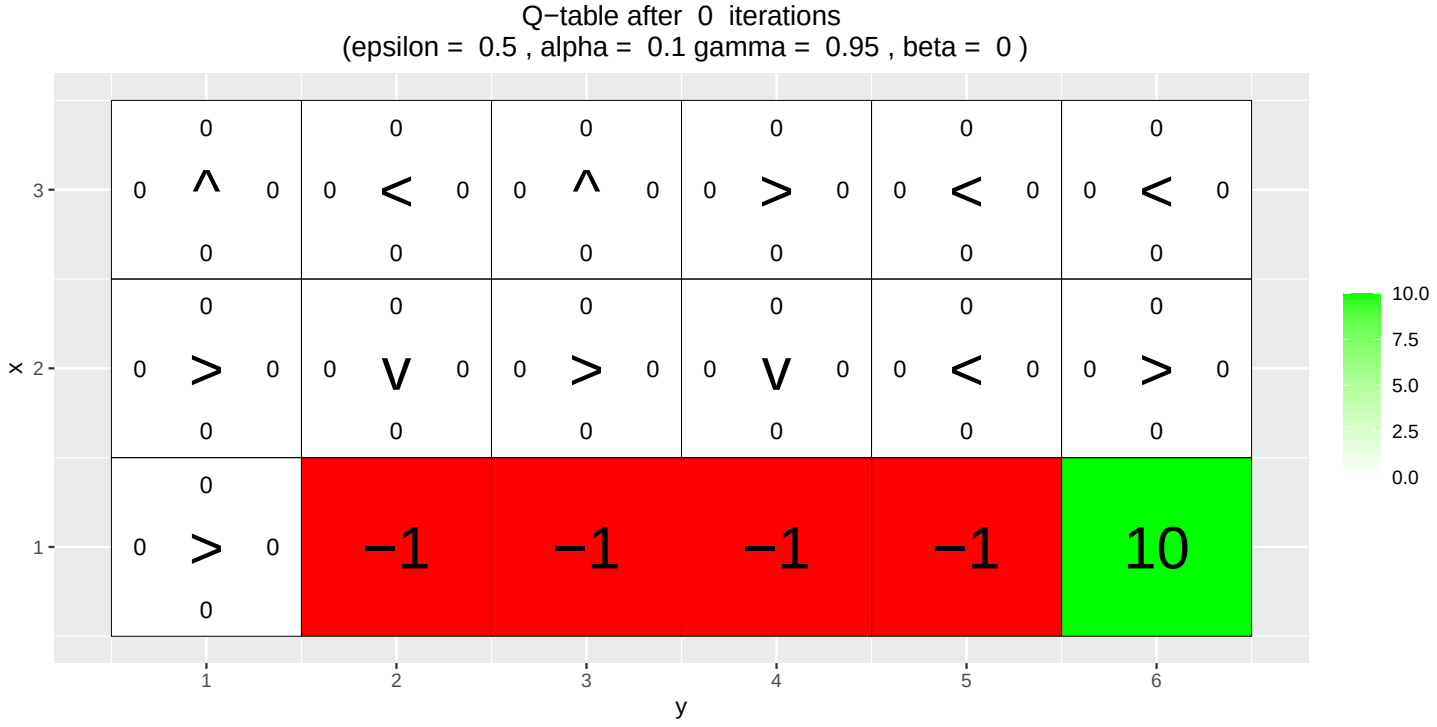

MovingAverage(reward, 100)
5.0
4.9
4.8
4.7
4.6
4.5
0
5000
10000
15000
20000
25000
30000
Index

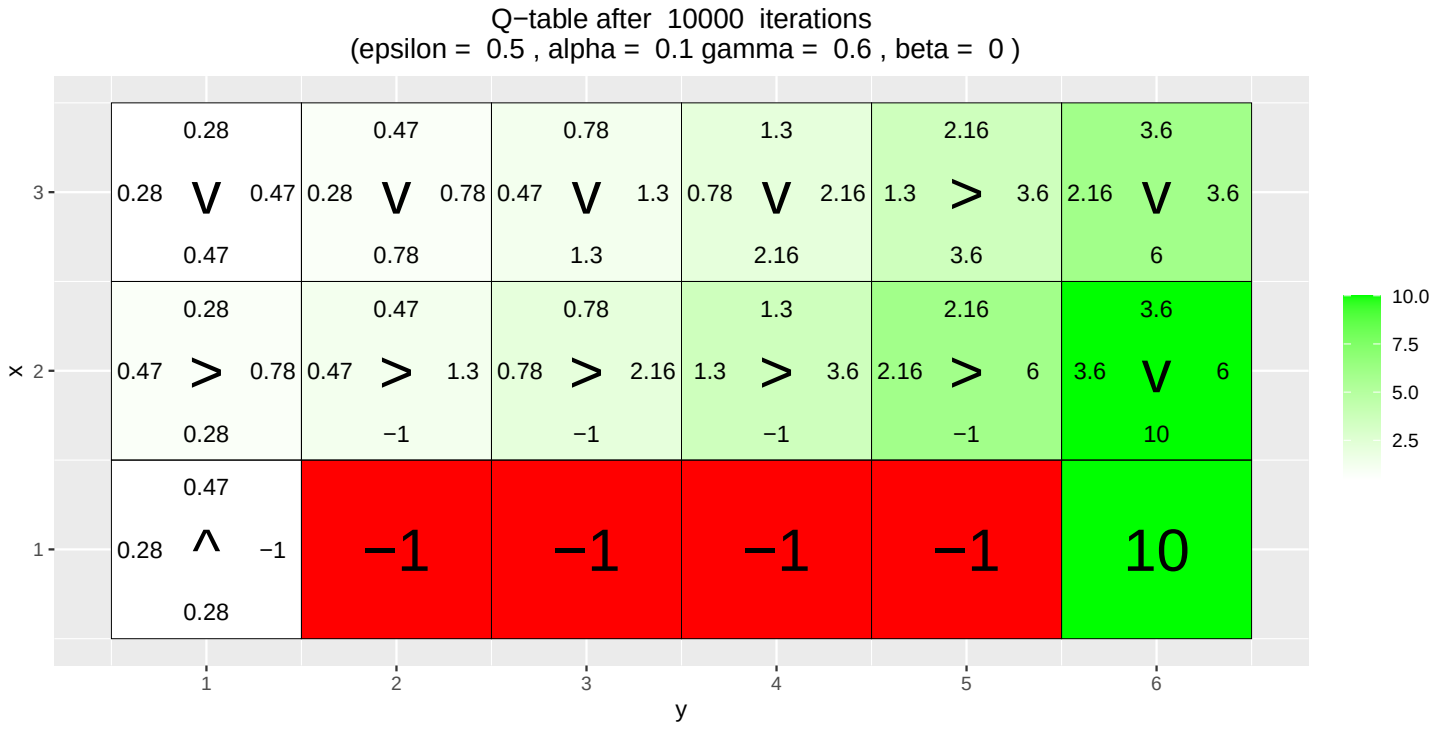


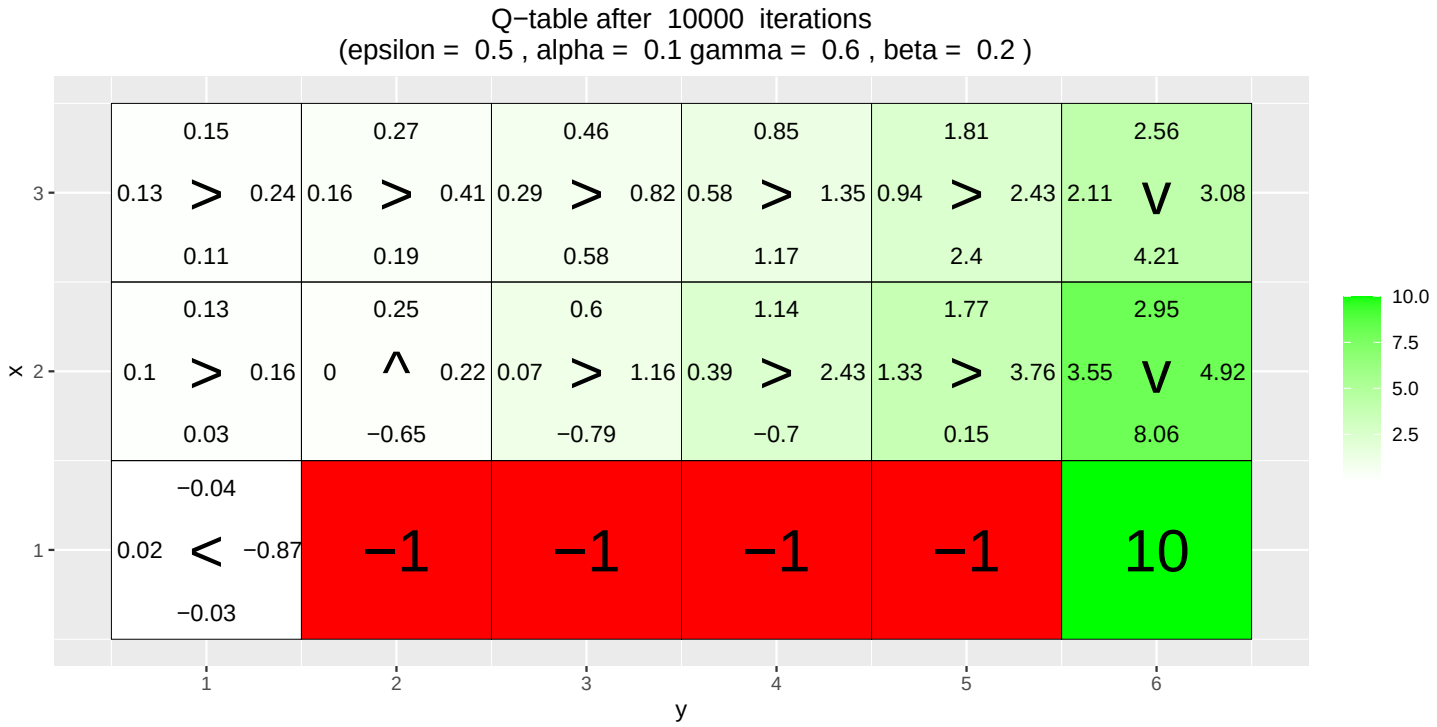
With epsilon = 0.1, it chooses randomly only 10% of the time. Finding the route to the 10 reward requires a lot of exploration around the easy way to reward 10. The agent learns the route to 5 and keeps taking it, and rarely ever reaches 10, leading to no or few experiences of that reward to propagate back through the Q-table.

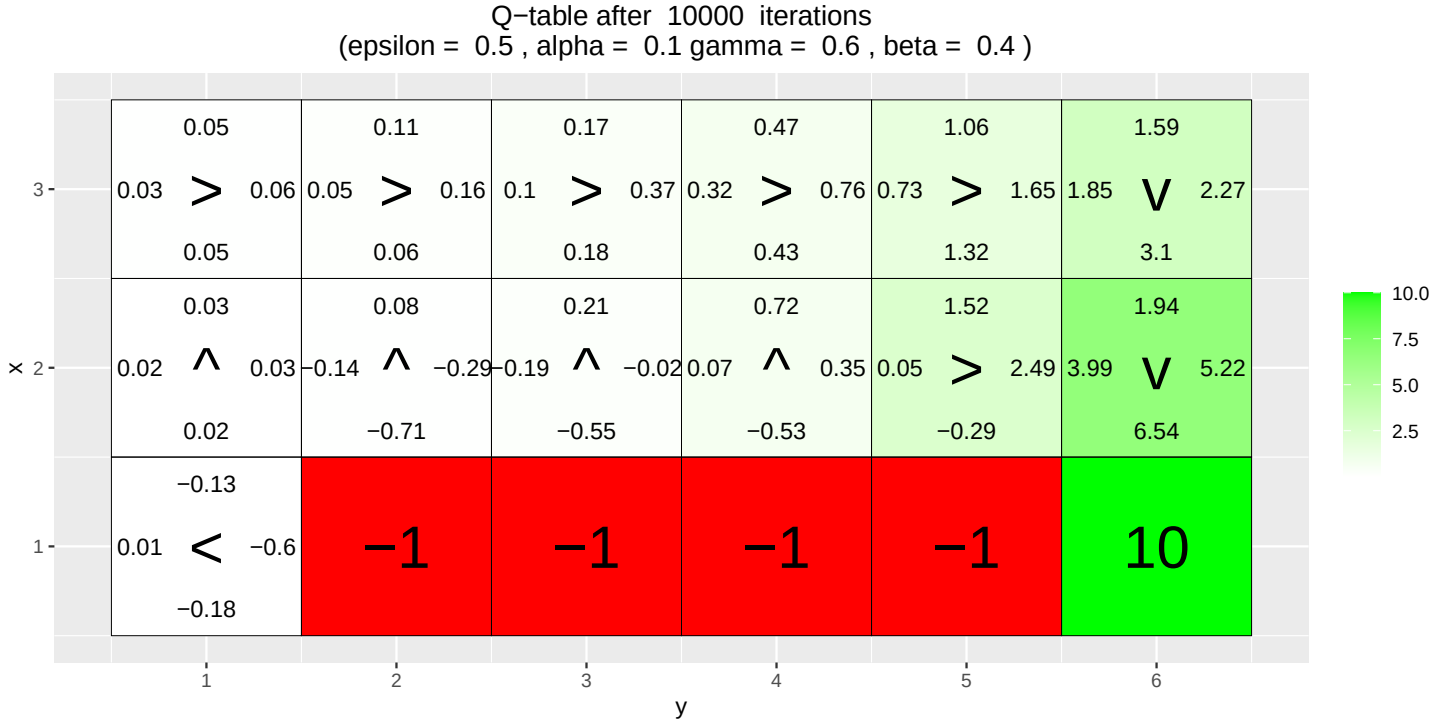
1.3 Environment C: effect of beta

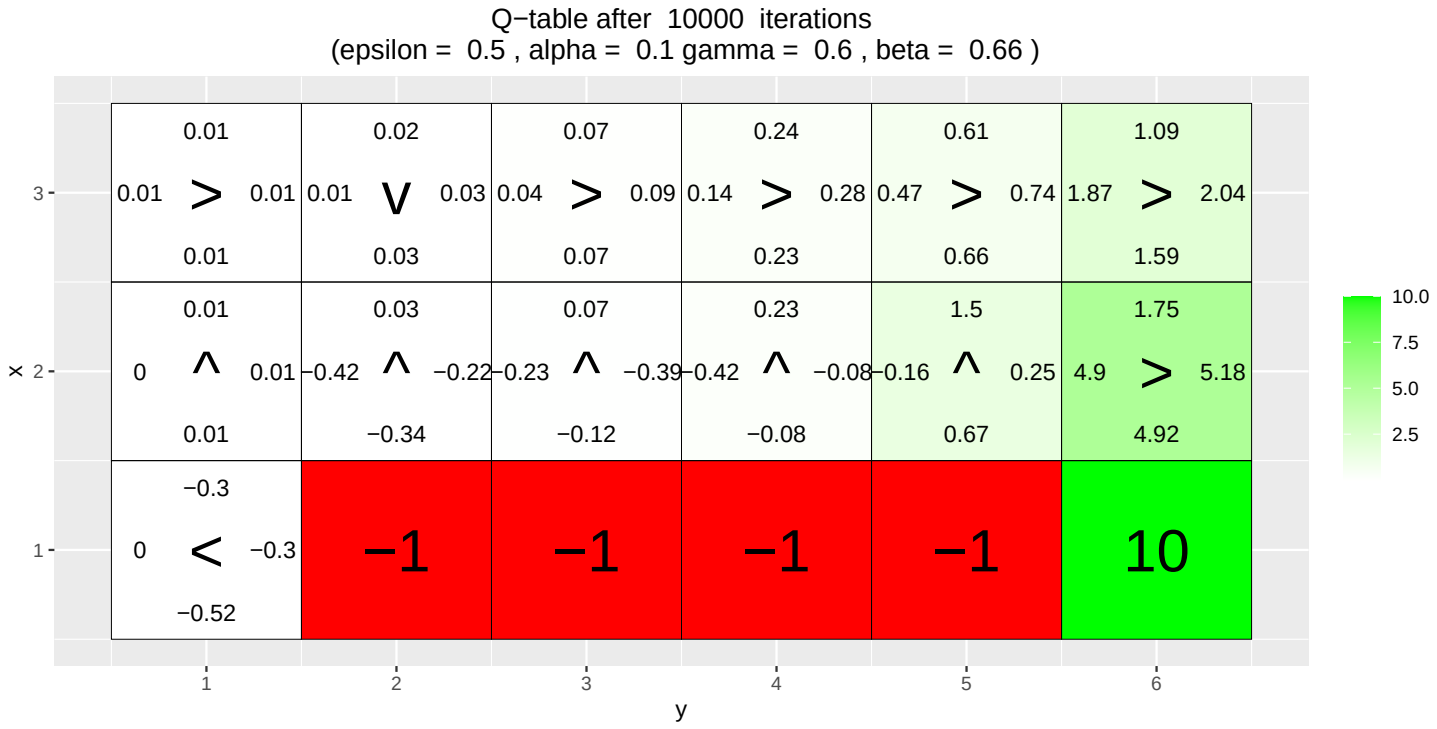
Environment C places a +10 reward beyond a sequence of negative-reward states. The experiment compares beta values 0, 0.2, 0.4 and 0.66, with epsilon 0.5 and gamma 0.6. Beta controls the probability of slipping either right or left of current state. Each side-slip has probability beta/2, while the intended move has probability 1-beta. As beta increases, movement becomes less predictable and reaching the goal becomes harder.






Q−table after 10000 iterations
(epsilon = 0.5 , alpha = 0.1 gamma = 0.6 , beta = 0.2)
0.15
0.13 > 0.24
0.11
0.27
0.16 > 0.41
0.19
0.46
0.29 > 0.82
0.58
0.85
0.58 > 1.35
1.17
1.81
0.94 > 2.43
2.4
2.56
2.11 V 3.08
4.21
0.13
0.1 > 0.16
0.03
0.25
0 ^ 0.22
−0.65
0.6
0.07 > 1.16
−0.79
1.14
0.39 > 2.43
−0.7
1.77
1.33 > 3.76
0.15
2.95
3.55 V 4.92
8.06
−0.04
0.02 < −0.87
−0.03
−1
−1
−1
−1
10
x
y
10.0
7.5
5.0
2.5


Q−table after 10000 iterations
(epsilon = 0.5 , alpha = 0.1 gamma = 0.6 , beta = 0.4)
0.05
0.03 > 0.06
0.05
0.11
0.05 > 0.16
0.06
0.17
0.1 > 0.37
0.18
0.47
0.32 > 0.76
0.43
1.06
0.73 > 1.65
1.32
1.59
1.85 V 2.27
3.1
0.03
0.02 ^ 0.03
0.02
0.08
−0.14 ^ −0.29
−0.71
0.21
−0.19 ^ −0.02
−0.55
0.72
0.07 ^ 0.35
−0.53
1.52
0.05 > 2.49
−0.29
1.94
3.99 V 5.22
6.54
−0.13
0.01 < −0.6
−0.18
−1
−1
−1
−1
10
x
y
10.0
7.5
5.0
2.5



2. REINFORCE

The REINFORCE implementation and its results are provided in `RL_Lab3_Colab.ipynb`.

2.1 Environment D: goals distributed across the grid

Eight goal positions are used for training and the other eight for validation.

Has the agent learned a good policy? Why or why not?

Reasonably good policy, but not reliable for every state and goal. In the validation plots, actions often point towards the goal, including unseen goals. This suggests that the network has learned to generalize. There are however some states that doesn't point toward the goal and might get caught in a loop. This is a result of the 20-step limit, training happens only if the goal is reached within 20 steps and the loop doesn't get a direct negative update.

Could Q-learning solve this task?

Yes, in principle, if we include the goal as part of the state. However, tabular Q-learning only learns the goal positions it experiences. In this environment we train on 8 goals and validation on the other 8, meaning it won't generalize on the unseen goals. We would need to train on them too.

2.2 Environment E: goals restricted to one row

Training goals are restricted to the top row, while validation goals are in other rows.

Has the agent learned a good policy? If the results differ from Environment D, why?

Poor policy. In this environment the agent only trains on goals whose first coordinate is 4 (top row). It gets examples of moving toward goals in that row, but no training signal for moving toward a goal on the rows below.

Appendix A: R Code

The Q-learning code is listed below, grouped by environment.

A.1 Q-learning functions

```r
# By Jose M. Peña and Joel Oskarsson.
# For teaching purposes.
# jose.m.pena@liu.se.

##########################################################################################################
# Q-learning
##########################################################################################################

# install.packages("ggplot2")
# install.packages("vctrs")
library(ggplot2)

arrows <- c("^", ">", "v", "<")
action_deltas <- list(c(1,0), # up
                      c(0,1), # right
                      c(-1,0), # down
                      c(0,-1)) # left

vis_environment <- function(iterations=0, epsilon = 0.5, alpha = 0.1, gamma = 0.95, beta = 0){

  # Visualize an environment with rewards.
  # Q-values for all actions are displayed on the edges of each tile.
  # The (greedy) policy for each state is also displayed.
  #
  # Args:
  #   iterations, epsilon, alpha, gamma, beta (optional): for the figure title.
  #   reward_map (global variable): a HxW array containing the reward given at each state.
  #   q_table (global variable): a HxWx4 array containing Q-values for each state-action pair.
  #   H, W (global variables): environment dimensions.

  df <- expand.grid(x=1:H,y=1:W)
  foo <- mapply(function(x,y) ifelse(reward_map[x,y] == 0,q_table[x,y,1],NA),df$x,df$y)
  df$val1 <- as.vector(round(foo, 2))
  foo <- mapply(function(x,y) ifelse(reward_map[x,y] == 0,q_table[x,y,2],NA),df$x,df$y)
  df$val2 <- as.vector(round(foo, 2))
  foo <- mapply(function(x,y) ifelse(reward_map[x,y] == 0,q_table[x,y,3],NA),df$x,df$y)
  df$val3 <- as.vector(round(foo, 2))
  foo <- mapply(function(x,y) ifelse(reward_map[x,y] == 0,q_table[x,y,4],NA),df$x,df$y)
  df$val4 <- as.vector(round(foo, 2))
  foo <- mapply(function(x,y)
    ifelse(reward_map[x,y] == 0,arrows[GreedyPolicy(x,y)],reward_map[x,y]),df$x,df$y)
  df$val5 <- as.vector(foo)
  foo <- mapply(function(x,y) ifelse(reward_map[x,y] == 0,max(q_table[x,y,]),
                                     ifelse(reward_map[x,y]<0,NA,reward_map[x,y])),df$x,df$y)
  df$val6 <- as.vector(foo)

  print(ggplot(df,aes(x = y,y = x)) +
          scale_fill_gradient(low = "white", high = "green", na.value = "red", name = "") +
          geom_tile(aes(fill=val6)) +
          geom_text(aes(label = val1),size = 4,nudge_y = .35,na.rm = TRUE) +
          geom_text(aes(label = val2),size = 4,nudge_x = .35,na.rm = TRUE) +
          geom_text(aes(label = val3),size = 4,nudge_y = -.35,na.rm = TRUE) +
          geom_text(aes(label = val4),size = 4,nudge_x = -.35,na.rm = TRUE) +
          geom_text(aes(label = val5),size = 10) +
          geom_tile(fill = 'transparent', colour = 'black') +
          ggtitle(paste("Q-table after ",iterations," iterations\n",
                        "(epsilon = ",epsilon,", alpha = ",alpha,"gamma = ",gamma,", beta = ",beta,")")) +
          theme(plot.title = element_text(hjust = 0.5)) +
          scale_x_continuous(breaks = c(1:W),labels = c(1:W)) +
```

```
          scale_y_continuous(breaks = c(1:H),labels = c(1:H)))

}

GreedyPolicy <- function(x, y){

  # Get a greedy action for state (x,y) from q_table.
  #
  # Args:
  #   x, y: state coordinates.
  #   q_table (global variable): a HxWx4 array containing Q-values for each state-action pair.
  #
  # Returns:
  #   An action, i.e. integer in {1,2,3,4}.

  # q_values for each action in given state
  q_values <- q_table[x, y, ]
  # Action(s) which has highest q_value
  best_actions <- which(q_values == max(q_values))
  # Select action at random from all greedy ties
  action <- best_actions[sample.int(length(best_actions), 1)]

}

EpsilonGreedyPolicy <- function(x, y, epsilon){

  # Get an epsilon-greedy action for state (x,y) from q_table.
  #
  # Args:
  #   x, y: state coordinates.
  #   epsilon: probability of acting randomly.
  #
  # Returns:
  #   An action, i.e. integer in {1,2,3,4}.

  # Explore with probability epsilon, else choose a greedy action.
  if(runif(1) < epsilon) {
    # Random action
    action <- sample.int(4, 1)
  } else {
    # Greedy policy
    action <- GreedyPolicy(x, y)
  }
  return(action)
}

transition_model <- function(x, y, action, beta){

  # Computes the new state after given action is taken. The agent will follow the action
  # with probability (1-beta) and slip to the right or left with probability beta/2 each.
  #
  # Args:
  #   x, y: state coordinates.
  #   action: which action the agent takes (in {1,2,3,4}).
  #   beta: probability of the agent slipping to the side when trying to move.
  #   H, W (global variables): environment dimensions.
  #
  # Returns:
  #   The new state after the action has been taken.
```

```
  delta <- sample(-1:1, size = 1, prob = c(0.5*beta,1-beta,0.5*beta))
  final_action <- ((action + delta + 3) %% 4) + 1
  foo <- c(x,y) + unlist(action_deltas[final_action])
  foo <- pmax(c(1,1),pmin(foo,c(H,W)))

  return (foo)
}

q_learning <- function(start_state, epsilon = 0.5, alpha = 0.1, gamma = 0.95,
                       beta = 0){

  # Perform one episode of Q-learning. The agent should move around in the
  # environment using the given transition model and update the Q-table.
  # The episode ends when the agent reaches a terminal state.
  #
  # Args:
  #   start_state: array with two entries, describing the starting position of the agent.
  #   epsilon (optional): probability of acting randomly.
  #   alpha (optional): learning rate.
  #   gamma (optional): discount factor.
  #   beta (optional): slipping factor.
  #   reward_map (global variable): a HxW array containing the reward given at each state.
  #   q_table (global variable): a HxWx4 array containing Q-values for each state-action pair.
  #
  # Returns:
  #   reward: reward received in the episode.
  #   correction: sum of the temporal difference correction terms over the episode.
  #   q_table (global variable): Recall that R passes arguments by value. So, q_table being
  #   a global variable can be modified with the superassigment operator <<-.

  # Your code here.
  state <- start_state
  episode_correction <- 0
  repeat{
    # Follow policy, execute action, get reward.
    action <- EpsilonGreedyPolicy(state[1], state[2], epsilon)
    new_state <- transition_model(state[1], state[2], action, beta)
    reward <- reward_map[new_state[1], new_state[2]]

    # q_value of current  state-action pair
    curr_value <- q_table[state[1], state[2], action]
    # q_value of best action in new state
    next_value <- max(q_table[new_state[1], new_state[2], ])
    # the transition "error" based on the reward plus the best estimated future value
    delta <- reward + gamma * next_value - curr_value

    # Q-table update by adding product of "error" and alpha.
    q_table[state[1], state[2], action] <<-
      q_table[state[1], state[2], action] + alpha * delta

    # episode correction accumulated
    episode_correction <- episode_correction + alpha * delta
    # update state
    state <- new_state

    if(reward!=0)
      # End episode.
      return (c(reward,episode_correction))
```

```
  }

}
```

A.2 Environment A

```
#########################################################################################################
# Q-Learning Environments
#########################################################################################################

# Environment A (learning)

H <- 5
W <- 7

reward_map <- matrix(0, nrow = H, ncol = W)
reward_map[3,6] <- 10
reward_map[2:4,3] <- -1

q_table <- array(0,dim = c(H,W,4))

vis_environment()

for(i in 1:10000){
  foo <- q_learning(start_state = c(3,1))

  if(any(i==c(10,100,1000,10000)))
    vis_environment(i)
}
```

A.3 Environment B: epsilon and gamma

```
# Environment B (the effect of epsilon and gamma)

H <- 7
W <- 8

reward_map <- matrix(0, nrow = H, ncol = W)
reward_map[1,] <- -1
reward_map[7,] <- -1
reward_map[4,5] <- 5
reward_map[4,8] <- 10

q_table <- array(0,dim = c(H,W,4))

vis_environment()

MovingAverage <- function(x, n){

  cx <- c(0,cumsum(x))
  rsum <- (cx[(n+1):length(cx)] - cx[1:(length(cx) - n)]) / n

  return (rsum)
}

for(j in c(0.5,0.75,0.95)){
  q_table <- array(0,dim = c(H,W,4))
  reward <- NULL
```

```
  correction <- NULL

  for(i in 1:30000){
    foo <- q_learning(gamma = j, start_state = c(4,1))
    reward <- c(reward,foo[1])
    correction <- c(correction,foo[2])
  }

  vis_environment(i, gamma = j)
  plot(MovingAverage(reward,100),type = "l")
  plot(MovingAverage(correction,100),type = "l")
}
```

```
for(j in c(0.5,0.75,0.95)){
  q_table <- array(0,dim = c(H,W,4))
  reward <- NULL
  correction <- NULL

  for(i in 1:30000){
    foo <- q_learning(epsilon = 0.1, gamma = j, start_state = c(4,1))
    reward <- c(reward,foo[1])
    correction <- c(correction,foo[2])
  }

  vis_environment(i, epsilon = 0.1, gamma = j)
  plot(MovingAverage(reward,100),type = "l")
  plot(MovingAverage(correction,100),type = "l")
}
```

A.4 Environment C: beta

```
# Environment C (the effect of beta).

H <- 3
W <- 6

reward_map <- matrix(0, nrow = H, ncol = W)
reward_map[1,2:5] <- -1
reward_map[1,6] <- 10

q_table <- array(0,dim = c(H,W,4))

vis_environment()

for(j in c(0,0.2,0.4,0.66)){
  q_table <- array(0,dim = c(H,W,4))

  for(i in 1:10000)
    foo <- q_learning(gamma = 0.6, beta = j, start_state = c(1,1))

  vis_environment(i, gamma = 0.6, beta = j)
}
```

A.5 REINFORCE code source

The REINFORCE implementation, training runs and precomputed validation figures for Environments D and E are in `RL_Lab3_Colab.ipynb`.